

Introduction to Parallel Computing

Parallel Architectures

Levels of parallelism

- Instruction level (e.g. FMA= fused multiply and add).
- Vector processing (e.g. data parallelism)
- Hyperthreading(e.g. 4 hardware threads/core for Intel KNL, 8 for PowerPC).
- Cores / processor (e.g. 18 for Intel Broadwell)
- Processors (or sockets) / node
- Processors + accelerators (e.g. CPU+GPU)
- Nodes in a system
- Many systems (e.g. Grid, Cloud)

To reach the maximum (*peak*) performance of a parallel computer,
all levels of parallelism need to be exploited.

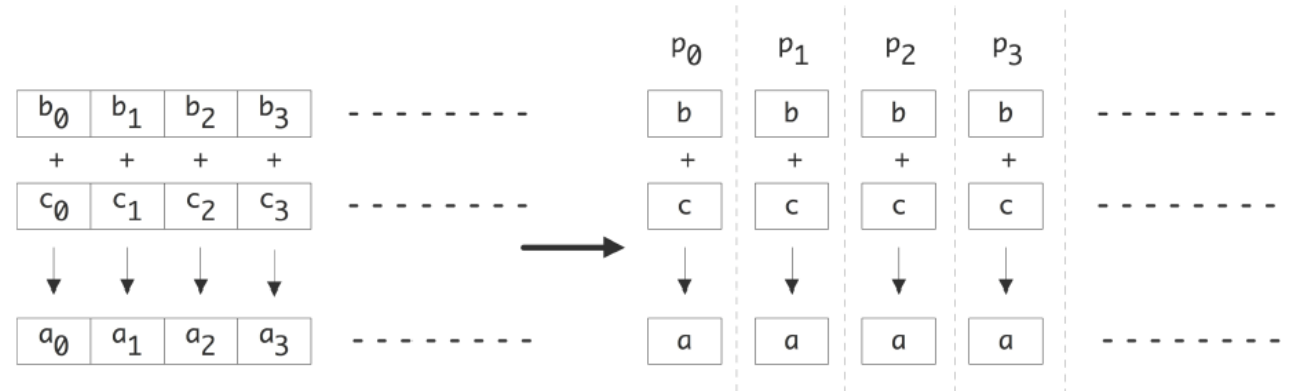
Introduction

- A general (and too simple) definition of parallel computers is «machines with more than one CPU that can be set to work on the same problem».
- But parallelism is hard to define precisely, since it can appear on several levels
 - inside a CPU several instructions can be ‘in flight’ simultaneously. Is the ILP, and it is outside explicit user control: it derives from the compiler and the CPU deciding which instructions can be processed simultaneously.
 - At the other extreme is the sort of parallelism where more than one instruction stream is handled by multiple processors. This type of parallelism is typically explicitly scheduled by the user.

Introduction

- In scientific (and also non scientific) codes, there is often a large amount of work to be done, and it is often regular to some extent, i.e. the same operation being performed on many data.
- If each operation takes a unit time, the original algorithm takes time t on n elements.
- The parallel execution on p processors takes, most of the time $t > t_p > n/p$, with $p \leq n$
- How can I write such application?

```
for (i=0; i<n; i++)  
    a[i] = b[i] + c[i];
```

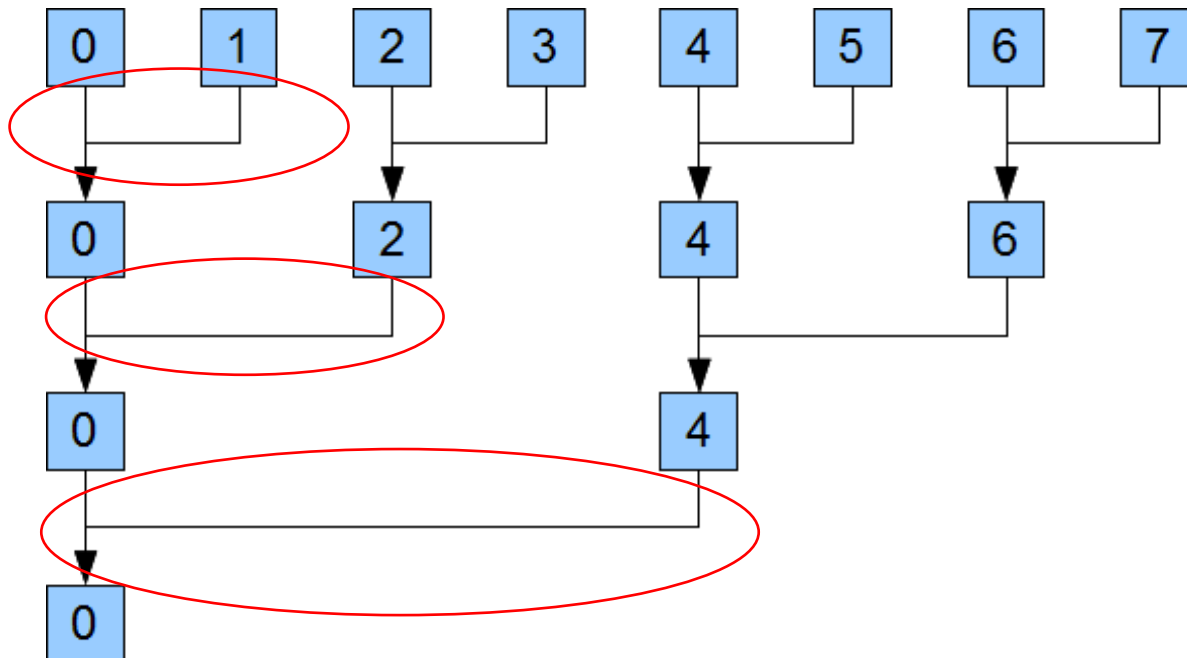


Parallel reduction

- Let us consider summing the elements of a vector, i.e. the reduction
- Seems difficult to parallelize in this form
- But

```
s = 0;  
for (i=0; i<n; i++)  
    s += x[i]
```

```
for (s=2; s<2*n; s*=2)  
    for (i=0; i<n-s/2; i+=s)  
        x[i] += x[i+s/2]
```



If $n = 8$ (and $p = 8$)

$S = 2, l = 0, < 7, += 2$ i.e. 0,2,4,6

$S = 4, l = 0, < 4, += 4$

$S = 8, l = 0, < 4, += 8$

$n/p * \log_2 n$

3 sums wrt 7

Introduction

Even from this simple example we can see some of the characteristics of parallel computing:

- Sometimes algorithms need to be rewritten to make them parallel
- A parallel algorithm may not show perfect speedup
- if each processors has its x_i (or even the full vector) in memory, there is the need to communicate data for the parallel reduction, not for the vector sum.

The prefix-sum problem

Given `int[] input`, produce `int[] output` where
`output[i]` is the sum of
`input[0]+input[1]+...+input[i]`

Sequential can be a CS1 exam problem:

```
int[] prefix_sum(int[] input) {  
    int[] output = new int[input.length];  
    output[0] = input[0];  
    for(int i=1; i < input.length; i++)  
        output[i] = output[i-1]+input[i];  
    return output;  
}
```

Does not seem parallelizable

- Work: $O(n)$, Span: $O(n)$
- This *algorithm* is sequential, but a *different algorithm* has Work: $O(n)$, Span: $O(\log n)$

Parallel prefix-sum

- The parallel-prefix algorithm does two passes
 - Each pass has $O(n)$ work and $O(\log n)$ span
 - So in total there is $O(n)$ work and $O(\log n)$ span
 - So like with array summing, parallelism is $n/\log n$
 - An exponential speedup
- First pass builds a tree bottom-up: the “up” pass
- Second pass traverses the tree top-down: the “down” pass

Historical note:

- Original algorithm due to R. Ladner and M. Fischer at the University of Washington in 1977

The algorithm, part 1

1. Up: Build a binary tree where
 - Root has sum of the range $[x, y)$
 - If a node has sum of $[lo, hi)$ and $hi > lo$,
 - Left child has sum of $[lo, middle)$
 - Right child has sum of $[middle, hi)$
 - A leaf has sum of $[i, i+1)$, i.e., `input[i]`

This is an easy fork-join computation: combine results by actually building a binary tree with all the range-sums

- Tree built bottom-up in parallel
- Could be more clever with an array like with heaps

Analysis: $O(n)$ work, $O(\log n)$ span

The algorithm, part 2

2. Down: Pass down a value **fromLeft**

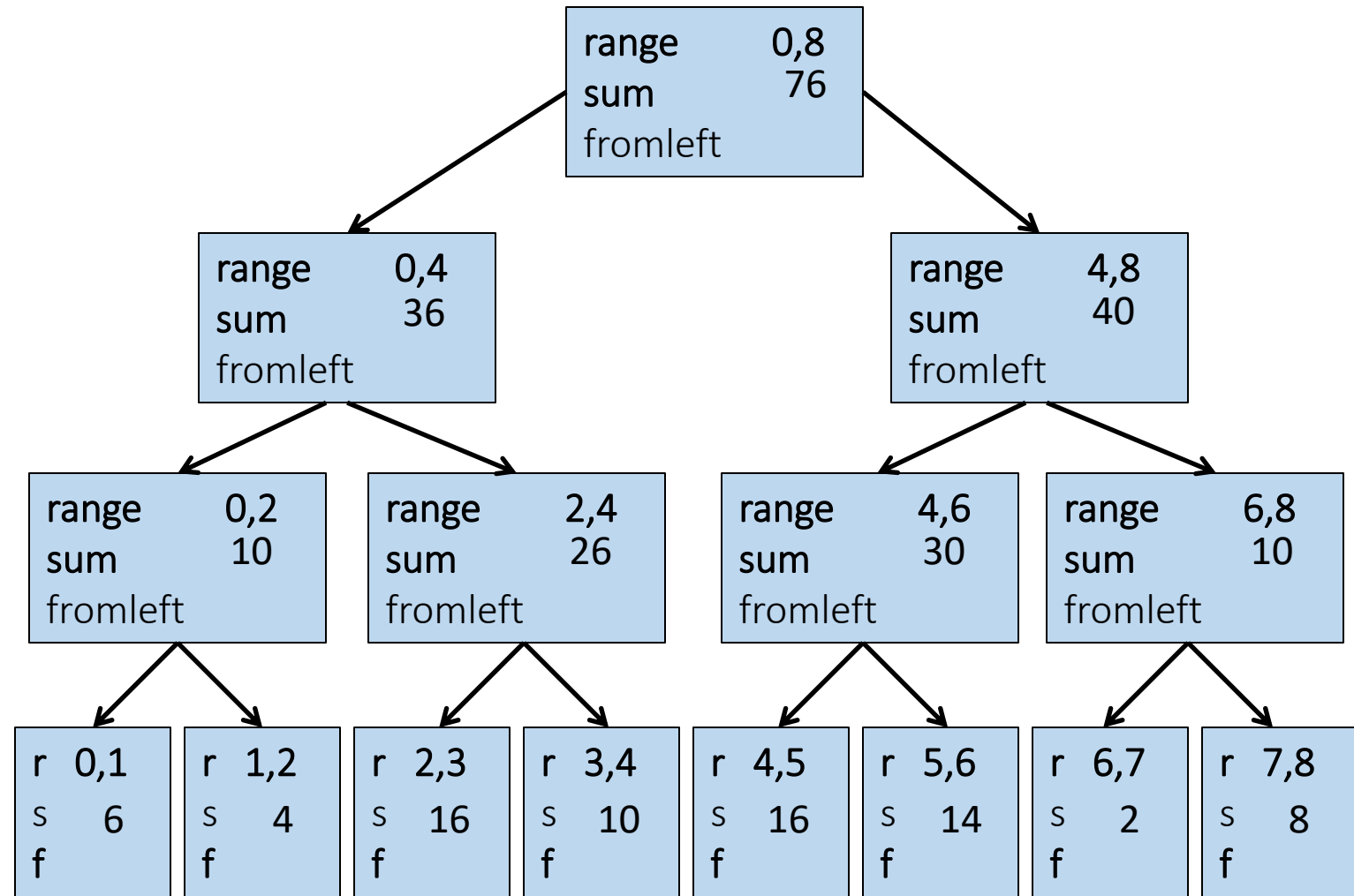
- Root given a **fromLeft** of 0
- Node takes its **fromLeft** value and
 - Passes its left child the same **fromLeft**
 - Passes its right child its **fromLeft** plus its left child's **sum** (as stored in part 1)
- At the leaf for array position **i**,
output[i] = fromLeft + input[i]

This is an easy fork-join computation: traverse the tree built in step 1 and produce no result

- Leaves assign to **output**
- Invariant: **fromLeft** is sum of elements left of the node's range

Analysis: $O(n)$ work, $O(\log n)$ span

Example



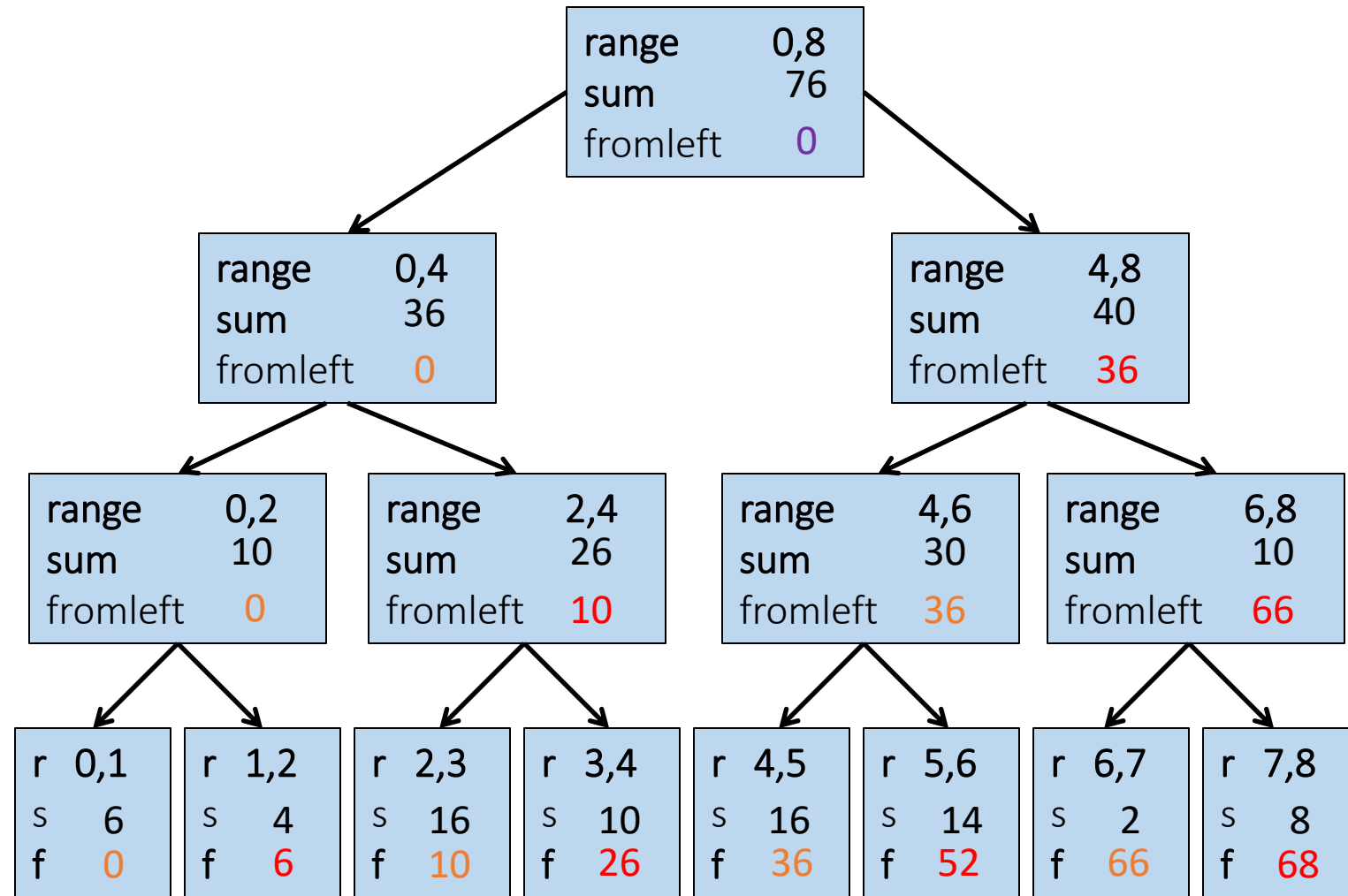
input

6	4	16	10	16	14	2	8
---	---	----	----	----	----	---	---

output

--	--	--	--	--	--	--	--

Example



input

6	4	16	10	16	14	2	8
---	---	----	----	----	----	---	---

output

6	10	26	36	52	66	68	76
---	----	----	----	----	----	----	----

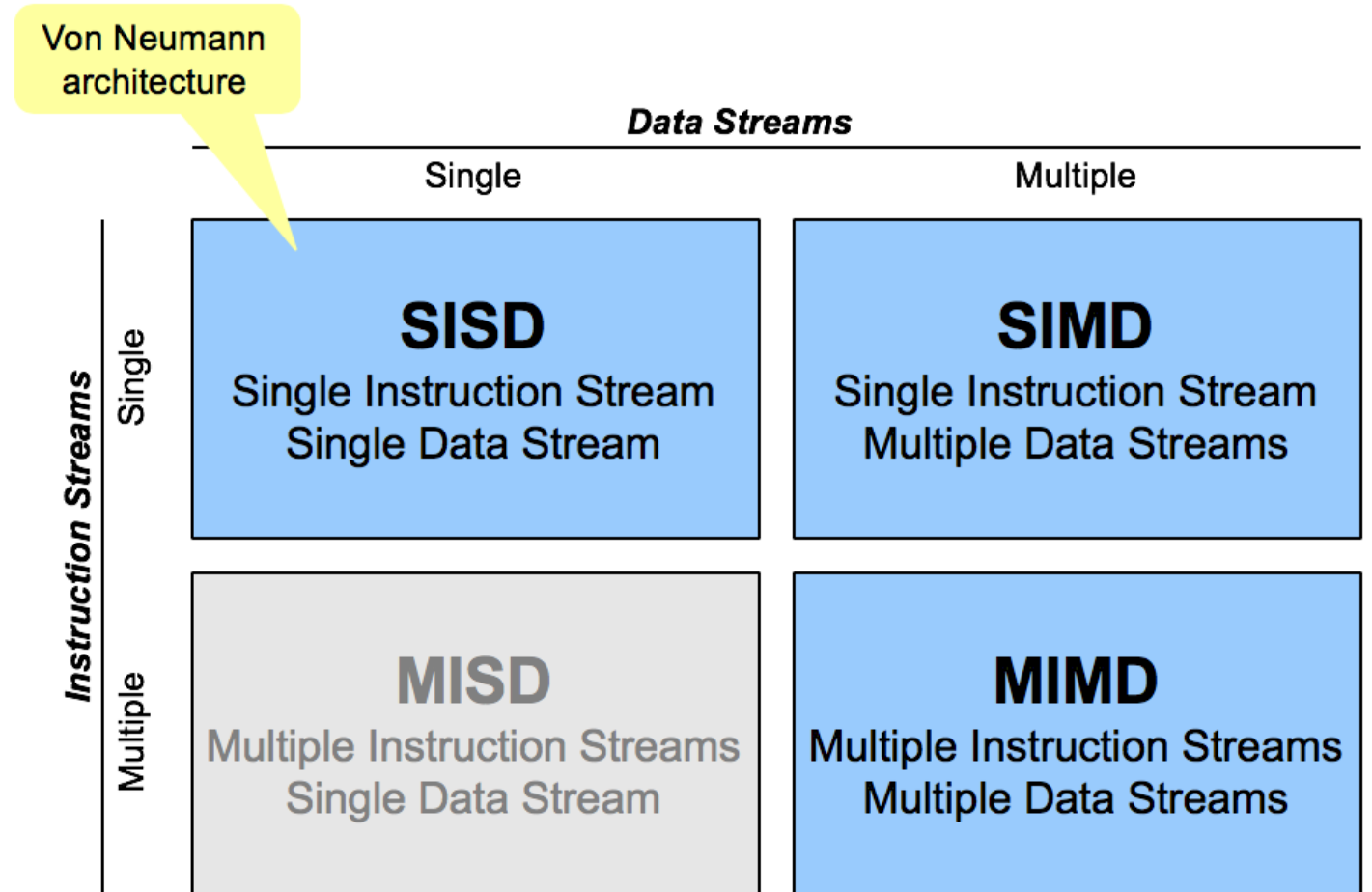
Which kind of parallelism

- **Data parallelism:** the same operation/instruction is applied in parallel to many data elements. This is a common scenario in scientific computing: parallelism often stems from the fact that a data set (vector, matrix, graph, . . .) is spread over many processors, each working on its part of the data.
- **Task parallelism:** the other cases, e.g. the **same a (sub)program** is applied in parallel to many data elements.
- We already saw ILP

```
...  
if CPU="a" then  
    do task "A"  
else if CPU="b" then  
    do task "B"  
end if  
...
```

Parallel Architectures

- Flynn's taxonomy distinguishes multi-processor computer architectures according to how they can be classified along the two independent dimensions of *Instruction Stream* and *Data Stream*.



- Other taxonomies, e.g.

<https://www.researchgate.net/publication/316511032> Parallel Computer Architectural Schemes

Why are parallel architectures important?

- There is no "typical" parallel computer: different vendors use different architectures
- See <https://www.top500.org/statistics/list/>
- There is currently no “universal” programming paradigm that fits all architectures
 - Parallel programs must be tailored to the underlying parallel architecture
 - The architecture of a parallel computer limits the choice of the programming paradigm that can be used (e.g. GPU vs distributed memory systems)

SIMD

```
double *A, *B, *C;  
for (int j=0; j<size; j++){  
    A[j] = B[j] + C[j];  
}
```

Register widths

- 1 operand



- 2 operands (SSE)



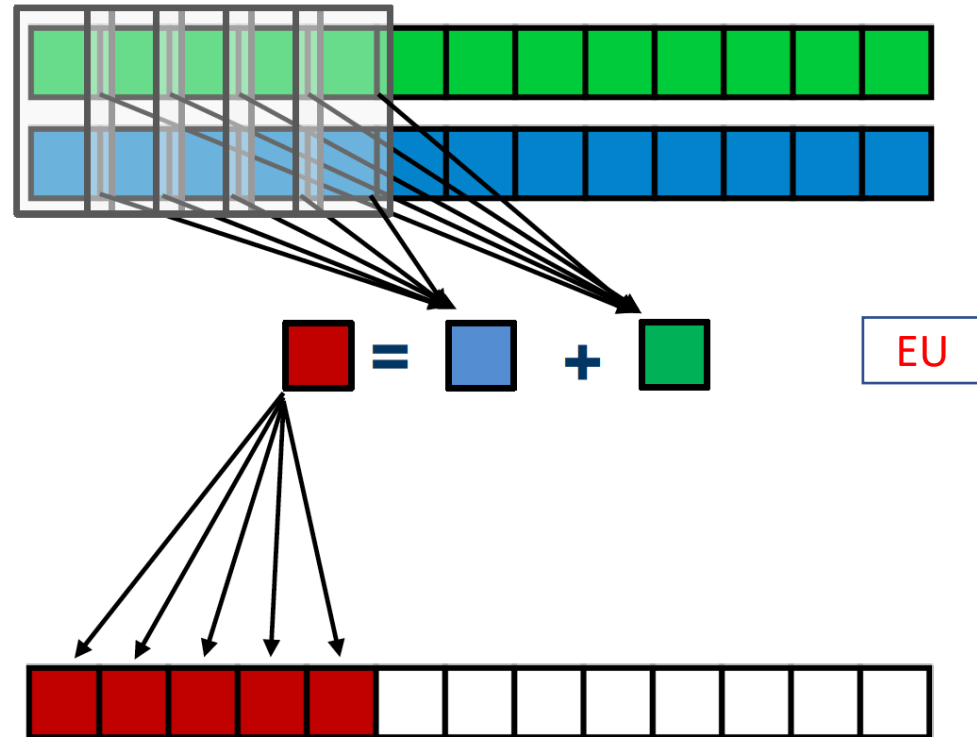
- 4 operands (AVX)



- 8 operands (AVX512)



Scalar execution



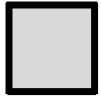
```
double *A, *B, *C;
for (int j=0; j<size; j++){
    A[j] = B[j] + C[j];
}
```

SIMD

SIMD execution

Register widths

- 1 operand



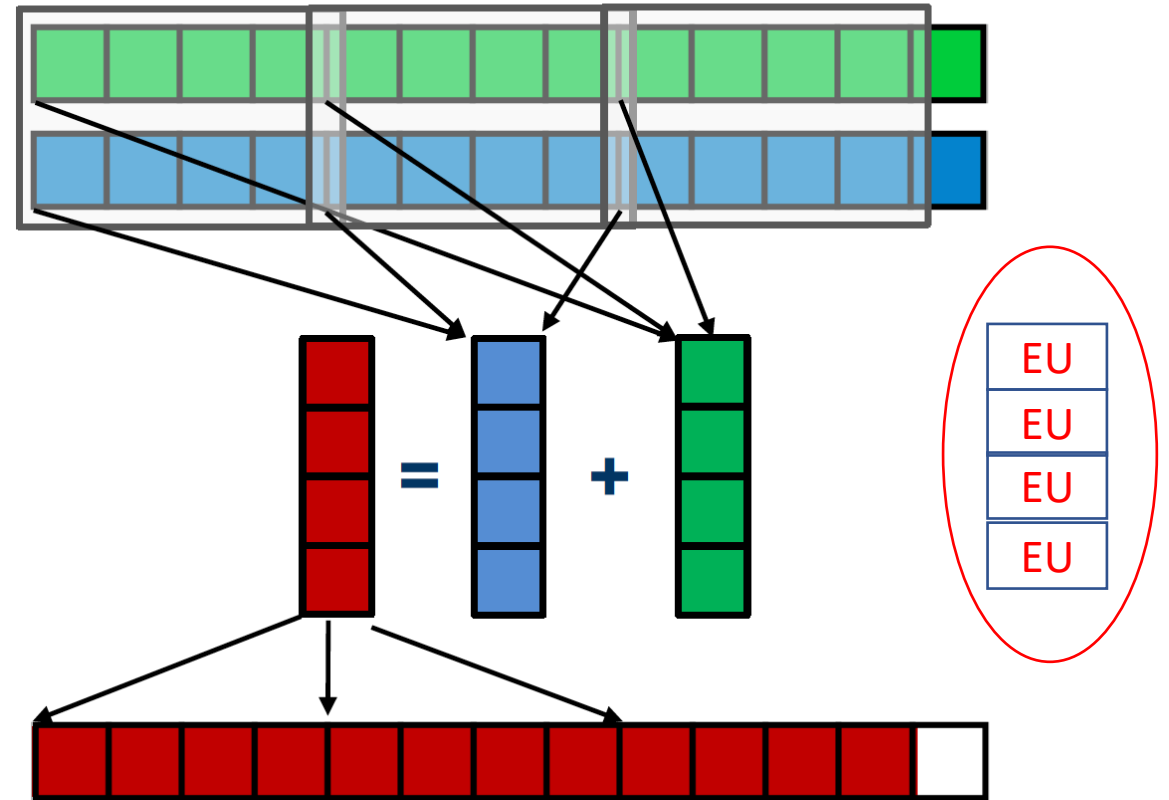
- 2 operands (SSE)



- 4 operands (AVX)



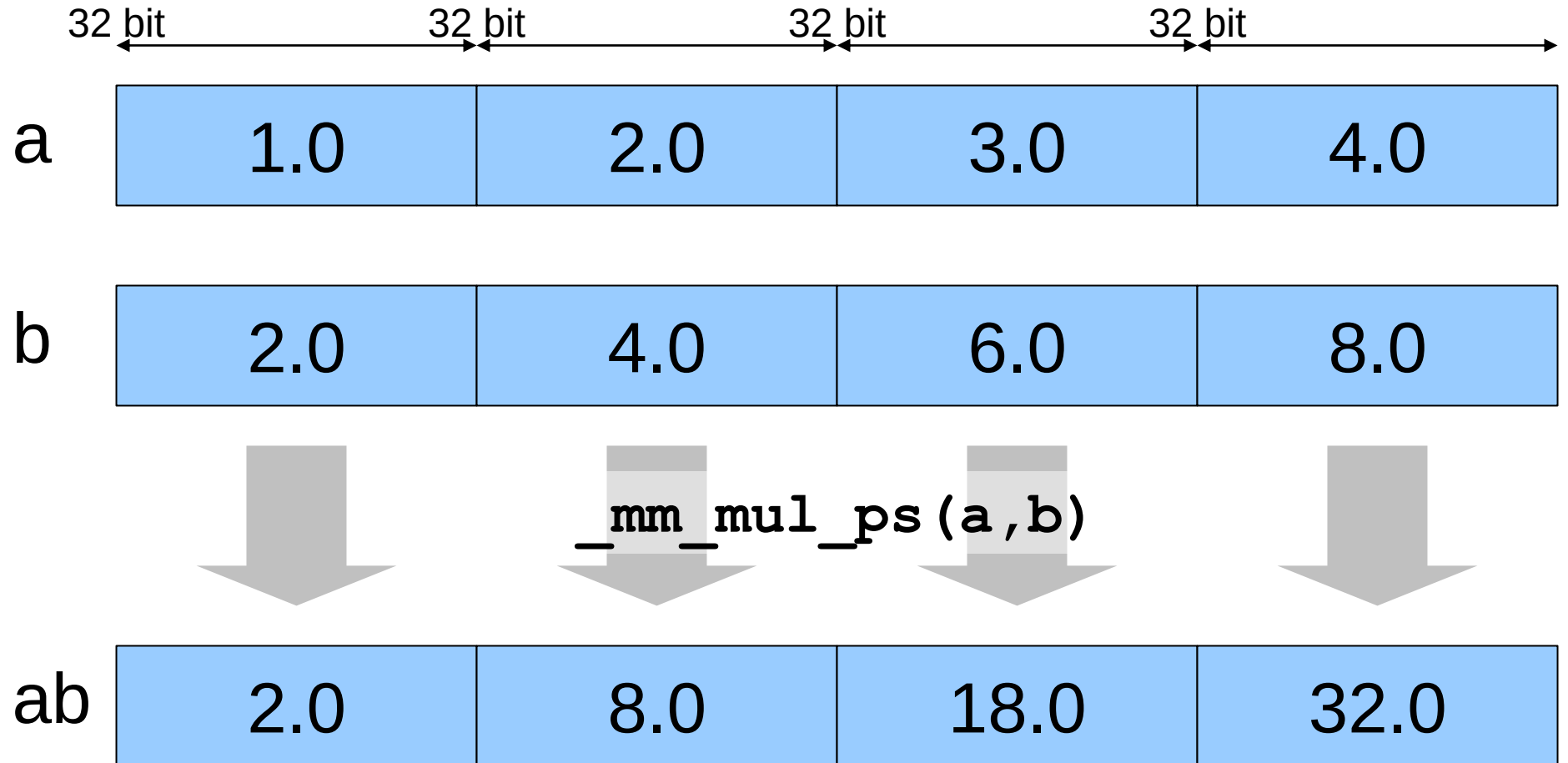
- 8 operands (AVX512)



Example

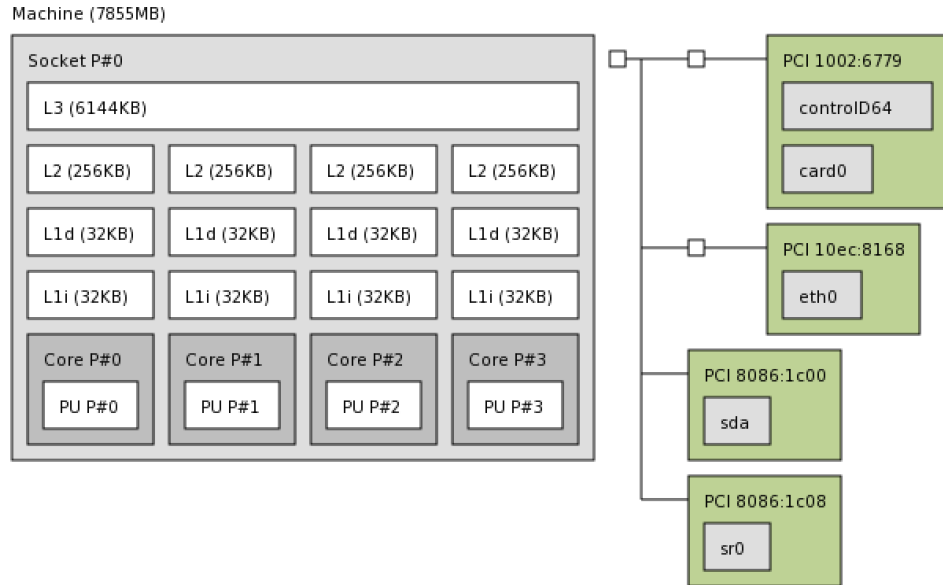
$a * b$

```
__m128 a = _mm_set_ps( 1.0, 2.0, 3.0, 4.0 );  
__m128 b = _mm_set_ps( 2.0, 4.0, 6.0, 8.0 );  
__m128 ab = _mm_mul_ps(a, b);
```



/proc/cpuinfo

<https://ark.intel.com/content/www/it/it/ark/products/52209/intel-core-i5-2500-processor-6m-cache-up-to-3-70-ghz.html>

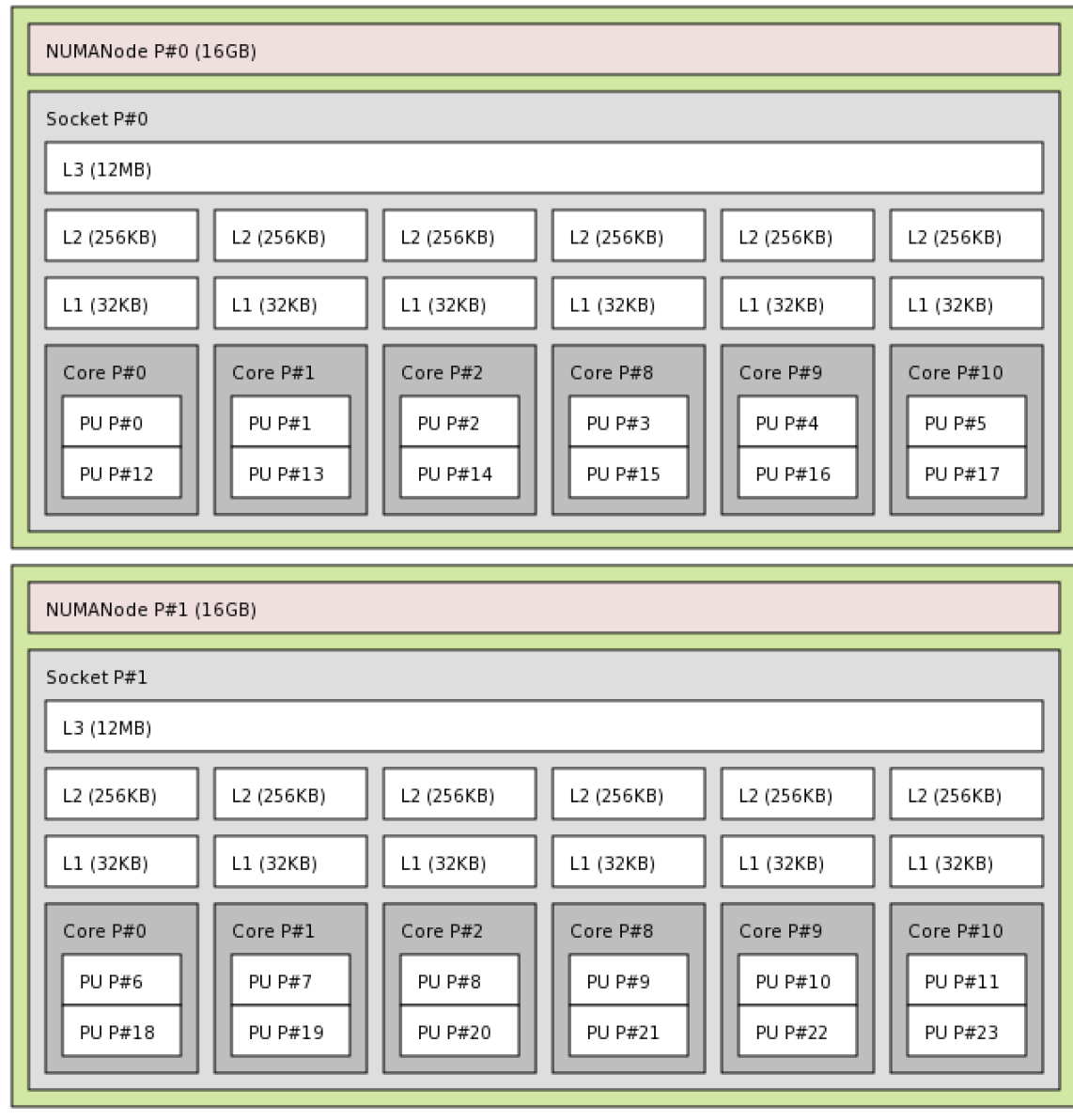


Cpuinfo, lstopo.... Many utilities to get the info
On Mac `sysctl -a | grep machdep.cpu`

```
[dago@girasole ~]$ cat /proc/cpuinfo
processor           : 0
vendor_id          : GenuineIntel
cpu family         : 6
model              : 42
model name         : Intel(R) Core(TM) i5-2500 CPU @ 3.30GHz
stepping           : 7
cpu MHz            : 1600.000
cache size         : 6144 KB
physical id        : 0
siblings           : 4
core id            : 0
cpu cores          : 4
apicid             : 0
initial apicid     : 0
fpu                : yes
fpu_exception      : yes
cpuid level        : 13
wp                 : yes
flags              : fpu vme de pse tsc msr pae mce cx8 apic sep mtrr pge mca
                    cmov pat pse36 clflush dts acpi mmx fxsr sse sse2 ss ht tm pbe syscall nx
                    rdtscp lm constant_tsc arch_perfmon pebs bts rep_good xtopology nonstop_t
                    sc aperfmperf pni pclmulqdq dtes64 monitor ds_cpl vmx smx est tm2 ssse3 cx
                    16 xtpr pdcm pcid sse4_1 sse4_2 x2apic popcnt tsc_deadline_timer aes xsave
                    avx lahf_lm ida arat epb xsaveopt pln pts dts tpr_shadow vnmi flexpriorit
                    y ept vpid
bogomips           : 6797.74
clflush size       : 64
cache_alignment    : 64
address sizes      : 36 bits physical, 48 bits virtual
power management:

processor           : 1
vendor_id          : GenuineIntel
```

Machine (32GB)



GPU, the “miracle” architecture

- Lots of hype about incredible speedup / high performance for low cost.
- Graphics processing: identical (and fairly simple) operations on lots of pixels
- Doesn't matter when any individual pixel gets processed, as long as they all get done in the end
- GPU is a SIMD engine
...and scientific computing is often very data--parallel

Tesla P100 – Pascal

- 5.3 TFLOPS for FP64 (double)
- 10.6 TFLOPS for FP32
- 21.2 TFLOPS for FP16

Less than 3 TF

<https://www.microway.com/knowledge-center-articles/detailed-specifications-of-the-cascade-lake-sp-intel-xeon-processor-scalable-family-cpus/>

Xeon Platinum 8280

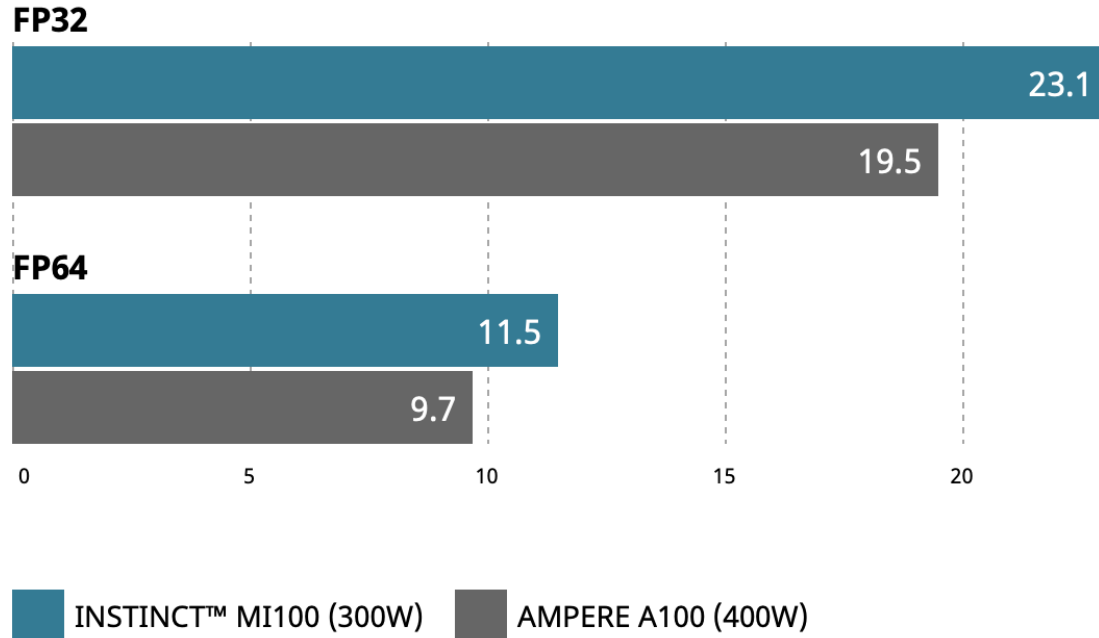


General Info

Designer	Intel
Manufacturer	Intel
Model Number	8280
Part Number	CD8069504228001
S-Spec	SRF9P
Market	Server
Introduction	April 2, 2019 (announced) April 2, 2019 (launched)
Release Price	\$10,009.00 (tray)

Today

(Peak TFLOPS) across range of mixed-precision compute¹

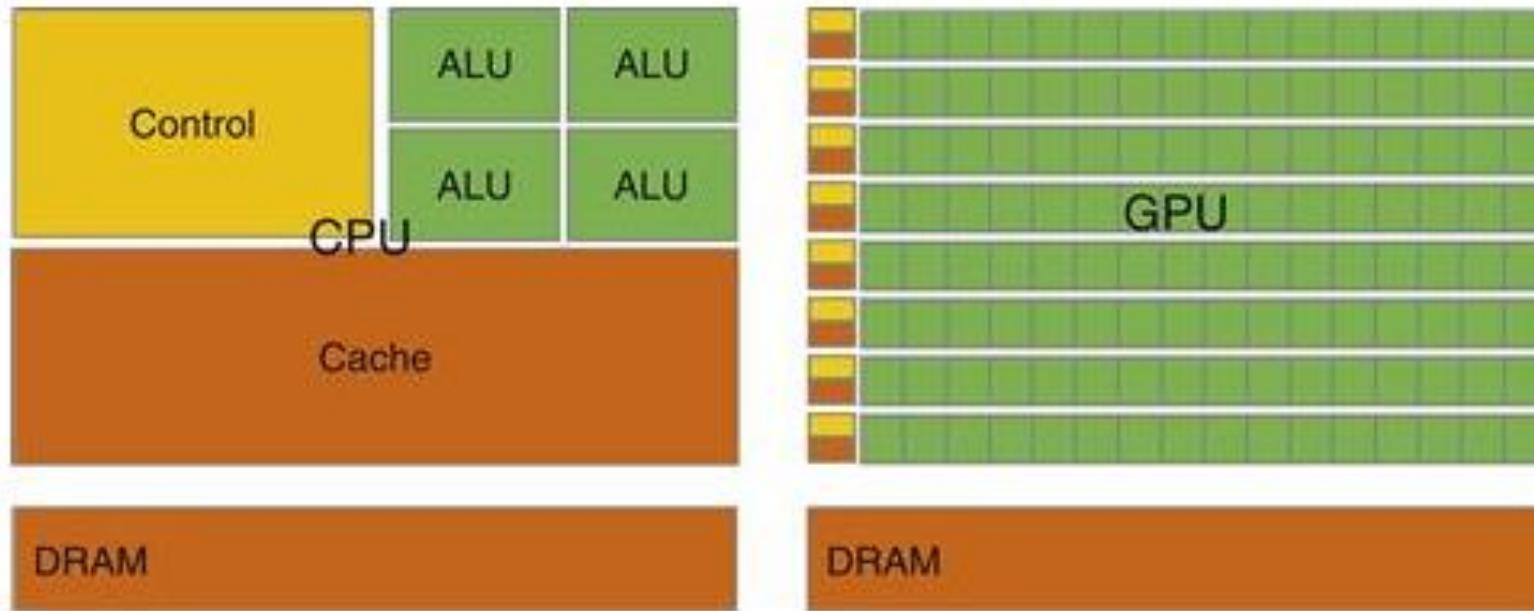


	A100 40GB PCIe	A100 80GB PCIe	A100 40GB SXM	A100 80GB SXM
FP64	9.7 TFLOPS			
FP64 Tensor Core	19.5 TFLOPS			
FP32	19.5 TFLOPS			
Tensor Float 32 (TF32)	156 TFLOPS 312 TFLOPS*			
BFLOAT16 Tensor Core	312 TFLOPS 624 TFLOPS*			
FP16 Tensor Core	312 TFLOPS 624 TFLOPS*			
INT8 Tensor Core	624 TOPS 1248 TOPS*			
GPU Memory	40GB HBM2	80GB HBM2e	40GB HBM2	80GB HBM2e
GPU Memory Bandwidth	1,555GB/s	1,935GB/s	1,555GB/s	2,039GB/s
Max Thermal Design Power (TDP)	250W	300W	400W	400W

<https://www.microway.com/knowledge-center-articles/detailed-specifications-of-the-ice-lake-sp-intel-xeon-processor-scalable-family-cpus/>

<https://www.intel.com/content/www/us/en/support/articles/000005755/processors.html>

GPUs

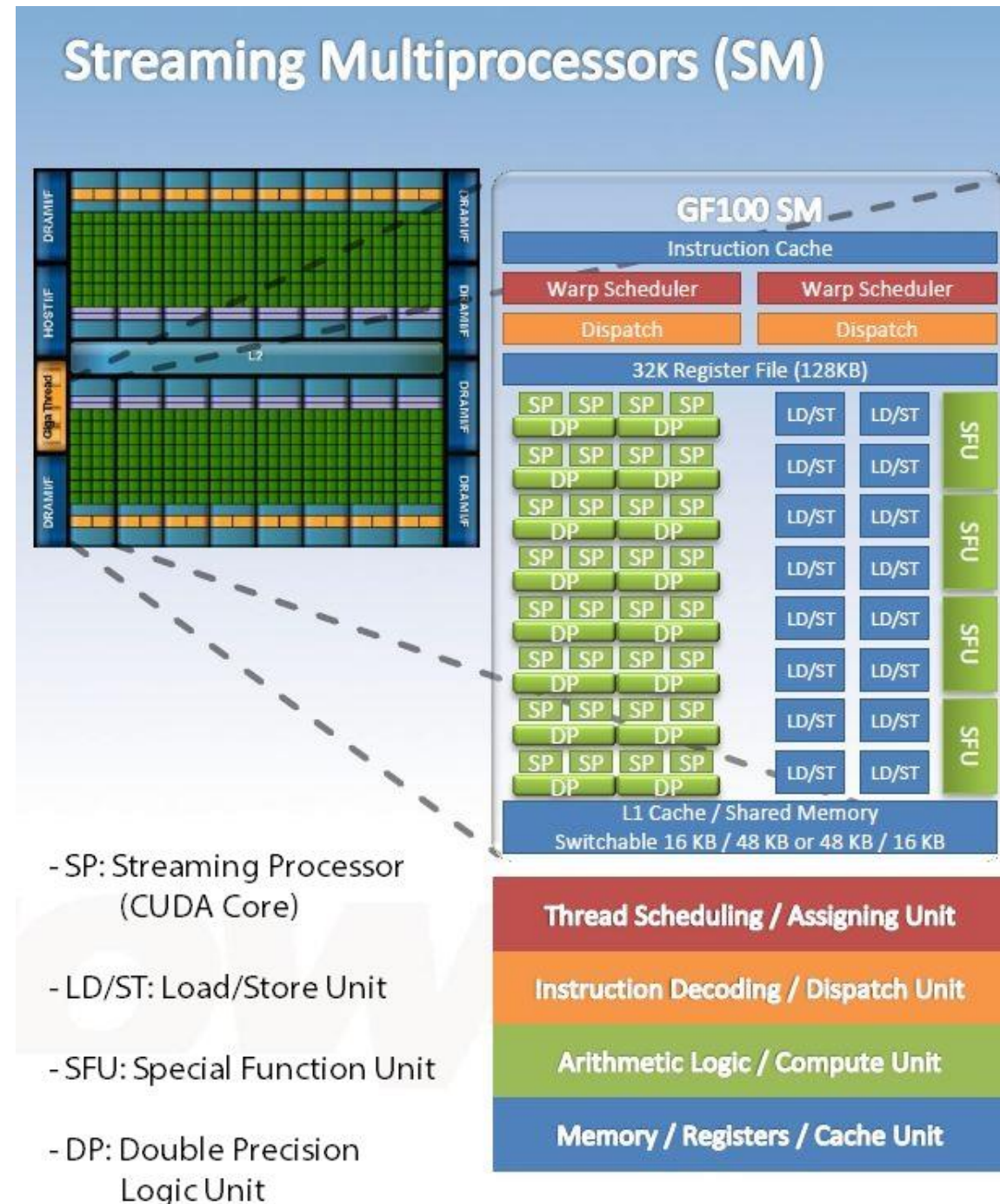


- GPU has a large number of «in order» cores;
- GPU cores **share control unit** (SIMD like style or better SIMT - Single Instruction Multiple Thread);
- GPU cores share small caches;
- GPU cores have lower frequency;

GPU is a (fine grained) parallel throughput-oriented computing engine

GPU core

- An SM contains a fetch/decode unit shared among multiple ALUs (the actual cores)
 - If there are 16 DP ALU, each instruction can operate on 16 DP values simultaneously
- Each GPU core maintains multiple execution contexts, and can switch between them at virtually zero cost
- A GPU with 16 SM x 16 DP ALU = 256 operations in parallel
- Details later in the course



Intel Xeon Phi Coprocessor

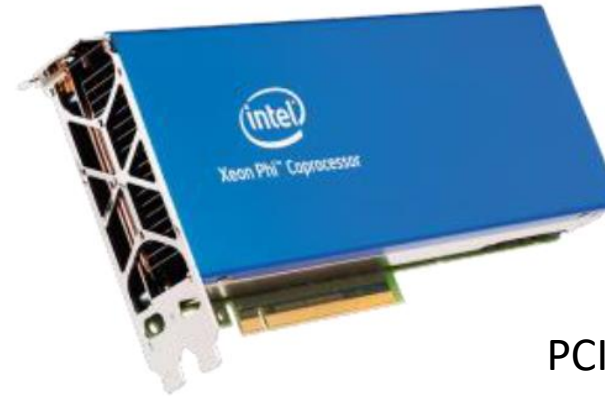
Low power device range based on Intel's Many Integrated Core (MIC) technology.

Large number of low frequency Pentium cores (e.g. 1.0 GHz) loosely connected on a chip with onboard memory.

The first commercially available Xeon Phi device Knight's Corner (KNC) could be used only as an accelerator.

Although runs standard FORTRAN and C/C++, difficult to obtain good performance.

Many application developers did not optimise codes for KNC.



PCI express connected device

Intel Knight's Corner

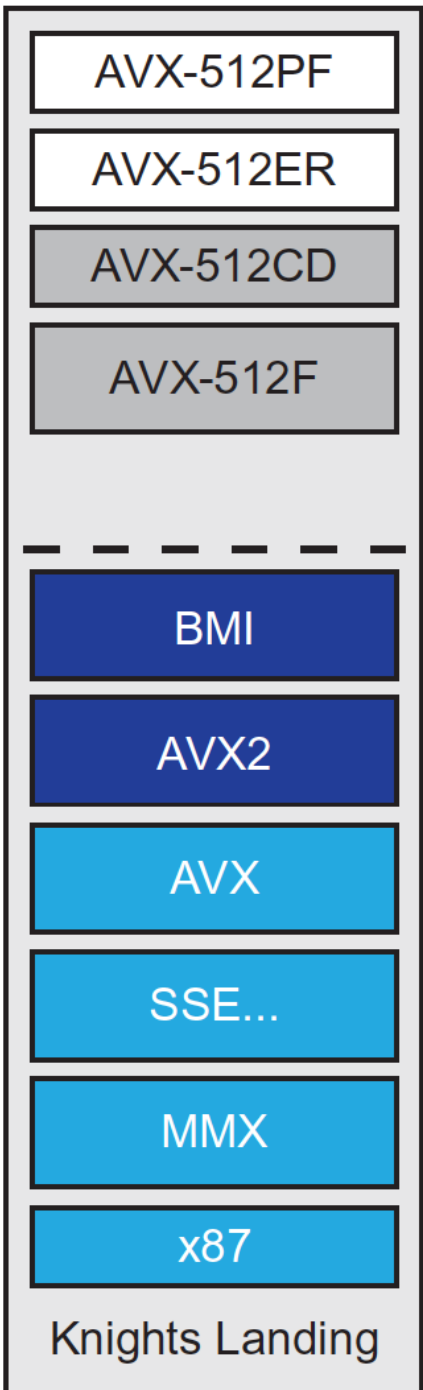
- 61 cores, 1.0-1.2 GHz
- 8-16 Gb RAM
- 512 bit vector unit
- 1-2 Tflops
- ring topology of cores
- With compiler option, runs standard FORTRAN or C (i.e. no CUDA or OpenCL necessary) and MPI.

Intel Xeon Phi Coprocessor

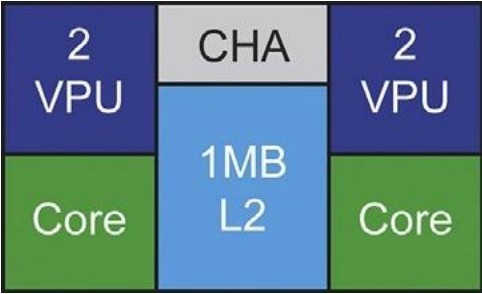
- The Intel Xeon Phi Coprocessor (IXPC) is designed to appear and operate more like a common multicore processor, though with many more cores. An IXPC has up to 61 cores, where each core implements **most** of the 64-bit x86 scalar instruction set. Instead of SSE and AVX instructions, an IXPC core has 512-bit vector instructions, which perform 16 single precision operations or eight double precision operations in a single instruction.
- Each core has a 32KB L1 instruction cache and 32KB L1 data cache, and a 512KB L2 cache. There is no cache shared across cores. The CPU clock is about 1.1GHz. With 61 cores and 16-wide vector operations, the IXPC has the equivalent of 976 GPU cores.
- The control unit can issue two instructions per clock, though instructions are issued in-order and those two instructions must be from the same thread (unlike Intel Hyperthreading). Each core stores the state of up to four threads, using multithreading to tolerate cache misses.
- The IXPC is packaged as an IO device, similar to a GPU. It has 8GB memory in current devices. The biggest potential advantage to the IXPC is its programmability using existing multi-core programming models. In many if not most cases, you really can just recompile your program and run it natively on an IXPC using MPI and/or OpenMP; however, to get good performance, you will likely have to tune or refactor your application.

Intel Xeon PHI -Knight's Landing (KNL)

- Is a standard Intel Architecture standalone processor
- It triples (3x) the performance of KNC, providing 3 Tflops DP and 6 Tflops SP peak performance
- Programming for Knights Landing is equivalent to programming an Intel® Xeon processor, i.e. both can execute the same binary executable programs
- ...but with extra attention on exploiting lots of parallelism because of the uniquely high degree of scaling possible with Knights Landing



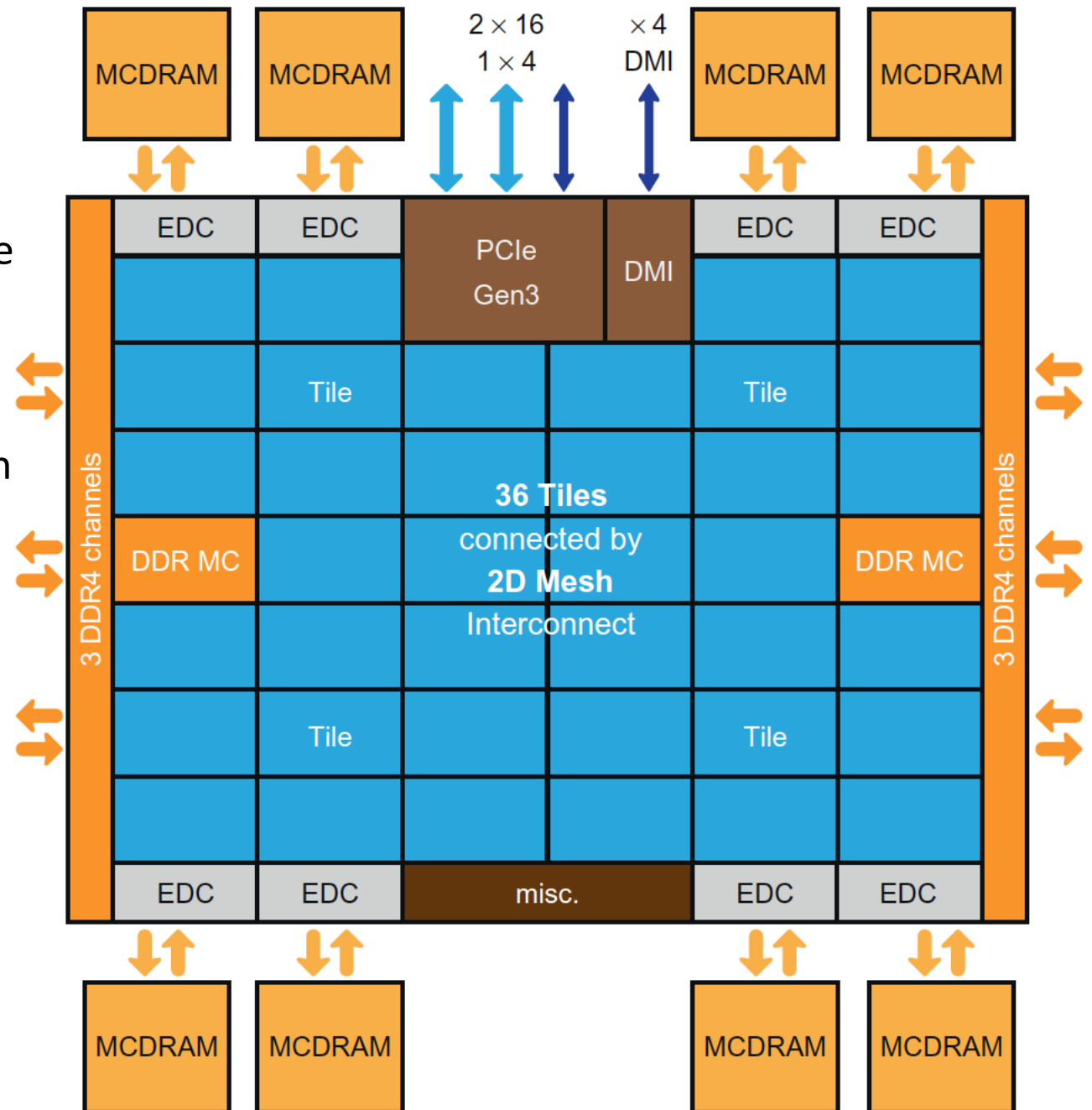
KNL Architecture



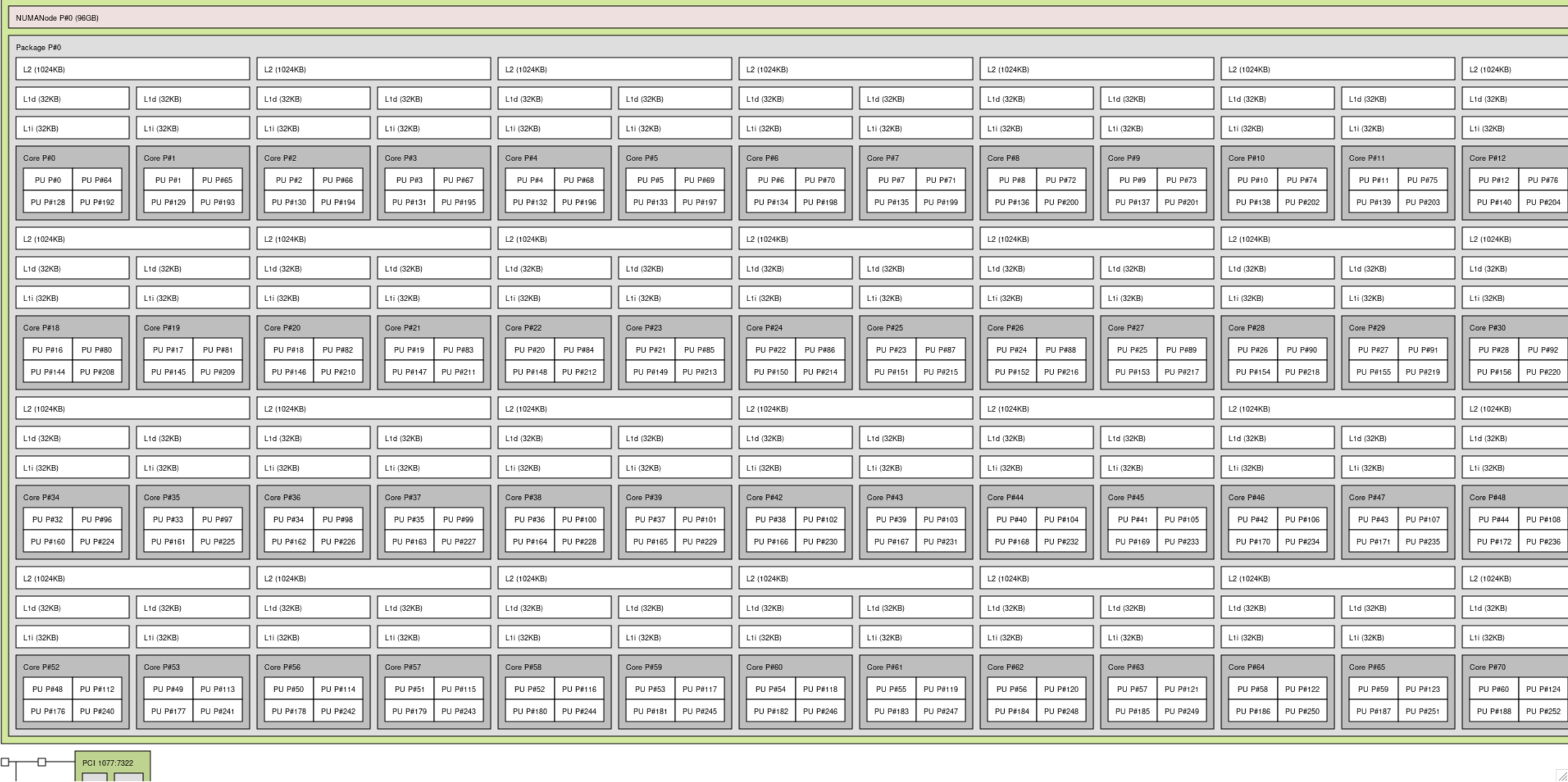
- Cores derived from the Intel Atom Silvermont
- Tile: 2 cores, 4 threads per core, 2 Vector processing units per core and 1 MB L2 cache shared among the cores.
- Up to 38 tiles, 36 active: 72 cores, 144 VPUs (32 tiles in our case)
- ISA: AVX-512 instructions, 8 DP (16 Flops) or 16 SP (32 Flops)
- AVX-512ER means exp and reciprocal.
 - A transcendental function is an analytic function that does not satisfy a polynomial equation. E.g. log, exp, trigonometric functions.
 - An intrinsic function is a function which the compiler implements directly when possible, rather than linking to a library-provided implementation

KNL Architecture

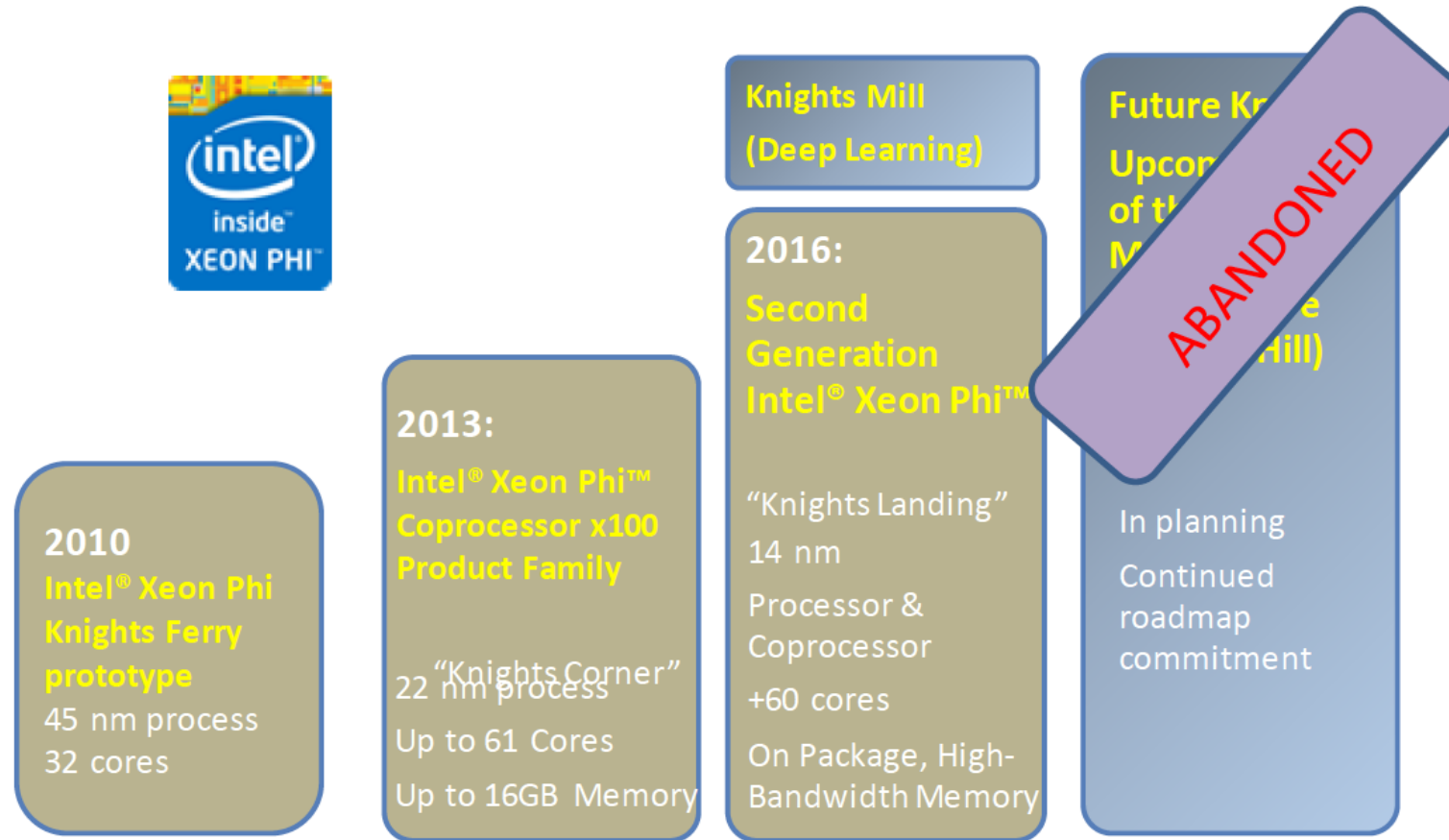
- MCDRAM: high bandwidth memory, 8 devices x 2 GB, with separate bus for R and W operations. The bandwidth from all the 8 MCDRAM is 450 GBps.
- MCDRAM can be used as RAM, as L3 cache for the RAM or in a hybrid way
- DRAM: 2 DDR4 memory controller, 3 channel each (1 DIMM up to 64 GB per channel). The aggregated bandwidth is 90 GBps
- On-die interconnect based on a ring architecture (4 rings) capable of 700 GBps of aggregated bandwidth
- YX routing: transaction always travels vertically first until it hits the target row, makes a turn, and then travels horizontally until its destination. A single hop on the mesh takes 2 clocks in the X-direction and only 1 in the Y-direction.



Machine (96GB total)



Intel Xeon PHI Roadmap



*Per Intel's announced products or planning process for future products

In November 2017 Intel announced that the Xeon Phi line would be abandoned due to “market and customer needs” (Intel).
Probably means that KNL had no market outside HPC.

CPU Flops

$$\text{FLOPS}_{\text{system}} = \frac{\text{instructions}}{\text{cycle}} \times \frac{\text{operations}}{\text{instruction}} \times \frac{\text{FLOPs}}{\text{operation}} \times \frac{\text{cycles}}{\text{second}} \times \frac{\text{cores}}{\text{node}} \times \frac{\text{nodes}}{\text{system}}$$

https://en.wikipedia.org/wiki/Xeon_Phi

KNL 7210:
64 core * 32 DP Flops * 1.3 GHz= 2.6 TFLOPS DP
8 DP * 2 EU * 2 FLOP (FMA = * and +)

SKU	CORES	BASE (GHz)	SINGLE CORE TURBO (GHz)	ALL CORE TURBO (GHz)	CACHE (MB)	TDP (Watts)
8380	40	2.3	3.4	3.0	60	270

3.8
TFLOPS

Microarchitecture	FLOPs			ISA
Intel Microarchitectures				
Core Penryn Nehalem	EUs	1 × 128-bit Multiplication + 1 × 128-bit Addition		SSE (128-bit)
	DP	4 FLOPs/cycle	2 FLOPs + 2 FLOPs	
	SP	8 FLOPs/cycle	4 FLOPs + 4 FLOPs	
Sandy Bridge Ivy Bridge	EUs	1 × 256-bit Multiplication + 1 × 256-bit Addition		AVX (256-bit)
	DP	8 FLOPs/cycle	4 FLOPs + 4 FLOPs	
	SP	16 FLOPs/cycle	8 FLOPs + 8 FLOPs	
Haswell Broadwell Skylake Kaby Lake Coffee Lake Whiskey Lake Amber Lake	EUs	2 × 256-bit FMA		AVX2 & FMA (256-bit)
	DP	16 FLOPs/cycle	2 × 8 FLOPs	
	SP	32 FLOPs/cycle	2 × 16 FLOPs	
Skylake (server)	EUs	2 × 512-bit FMA (varies by SKU)		AVX-512 & FMA (512-bit)
	DP	32 FLOPs/cycle	2 × 16 FLOPs	
	SP	64 FLOPs/cycle	2 × 32 FLOPs	
Intel MIC Microarchitectures				
Knights Landing	EUs	2 × 512-bit FMA (varies by SKU)		AVX-512 & FMA (512-bit)
	DP	32 FLOPs/cycle	2 × 16 FLOPs	
	SP	64 FLOPs/cycle	2 × 32 FLOPs	

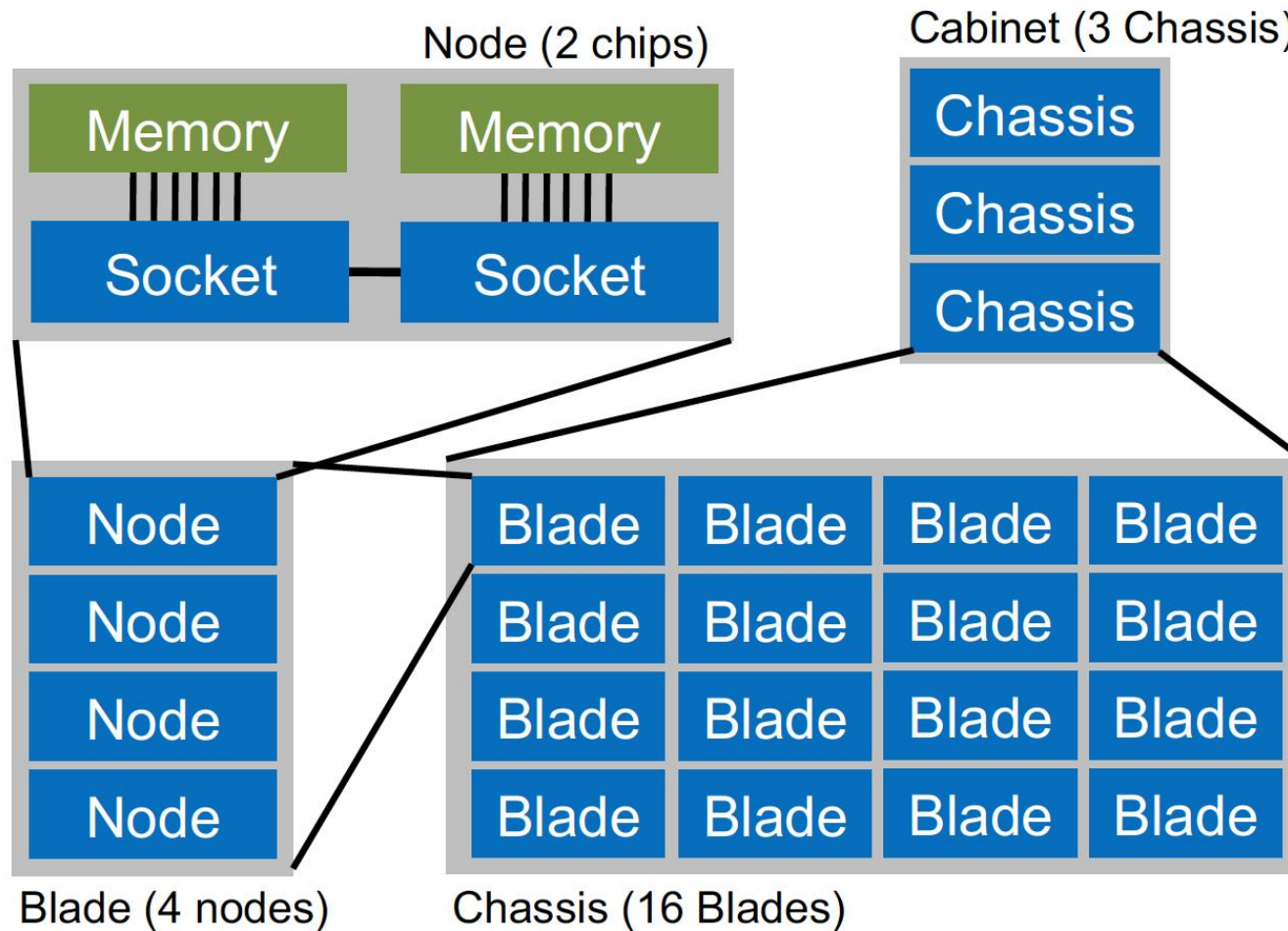
MIMD (or SPMD)

- **Multiple Instruction:** every processor may be executing a different instruction stream
- **Multiple Data:** every processor may be working with a different data stream
- Execution can be synchronous or asynchronous, deterministic or non-deterministic. Currently, the most common type of parallel computer/supercomputer /systems
- In practice **SPMD**: Single Program Multiple Data:
 - all processors execute the same code, not the same instruction
 - different control flow (i.e. branches, loop)
 - possible different amount of data, i.e. load unbalance

MIMD

Supercomputers: the most powerful computers available in a given period of time.

Powerful is meant in terms of execution speed, memory capacity and accuracy of the machine.



SuperMUC © LRZ

A HPC System consists of **many Cabinets!**

ENIAC (1947)

\$21,900 yearly electric bill
500 FLOPS



iPad (2013)

\$1.36 yearly electric bill
76,800,000,000 FLOPS



Performance:

↑ 153,599,999%

Energy Usage:

↓ 99.994%



IEEE

SANDIA ASCI RED

Date:

1996

Peak performance:

1.8Teraflops

Floor space:

150m

Power consumption:

800.000 Watt

Sony

PLAYSTATION 3

Date:

2006

Peak performance:

>1.8Teraflops

Floor space:

0.08m

Power consumption:

<200 Watt

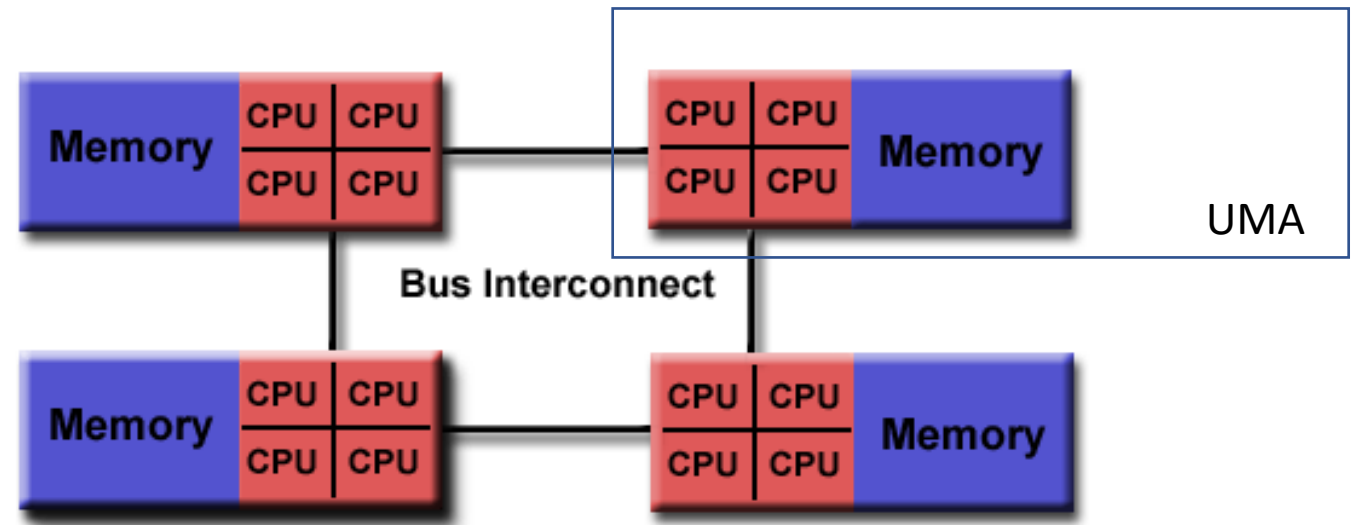
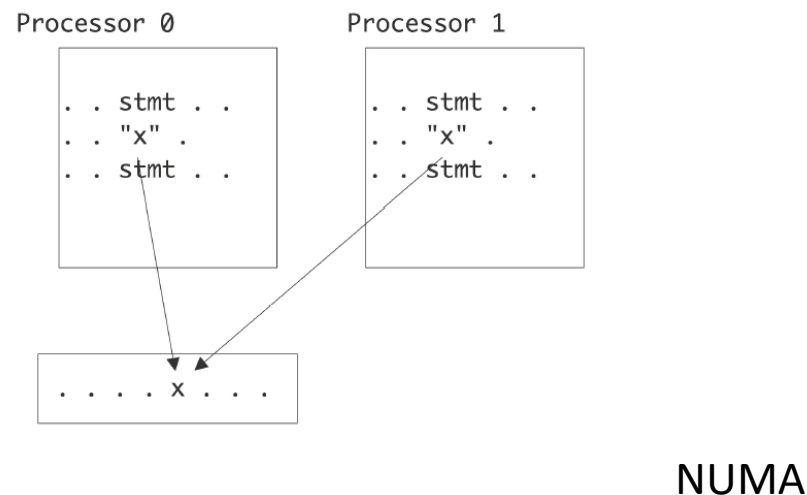


<https://phys.org/news/2010-12-air-playstation-3s-supercomputer.html>

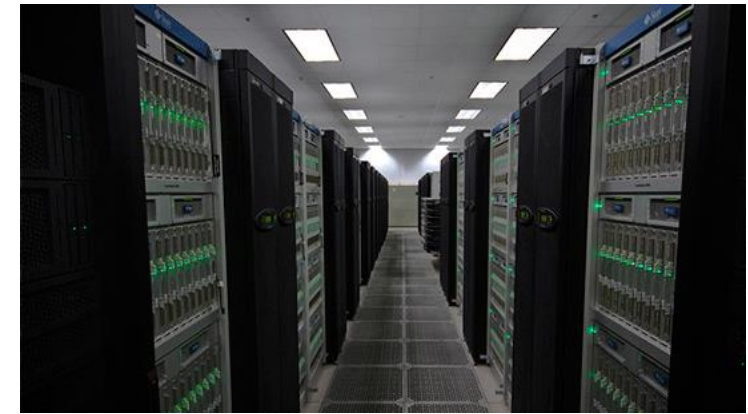
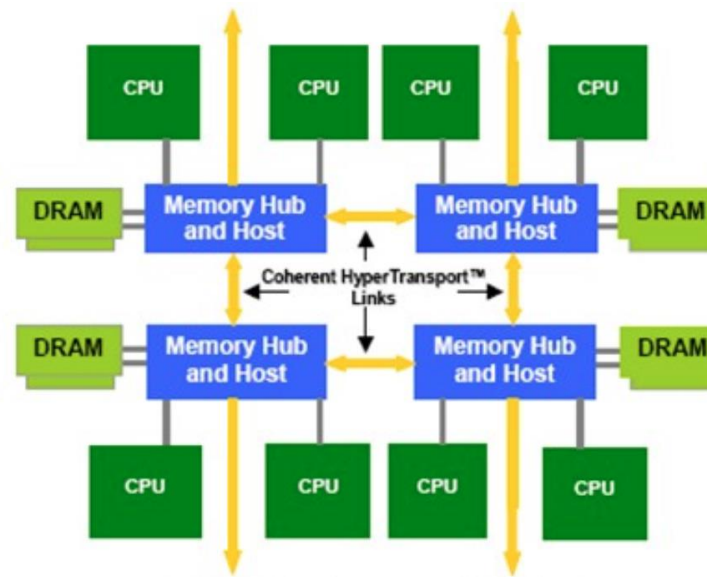
Yesterday's supercomputers could become today's appliances

Memory Access – Shared Memory

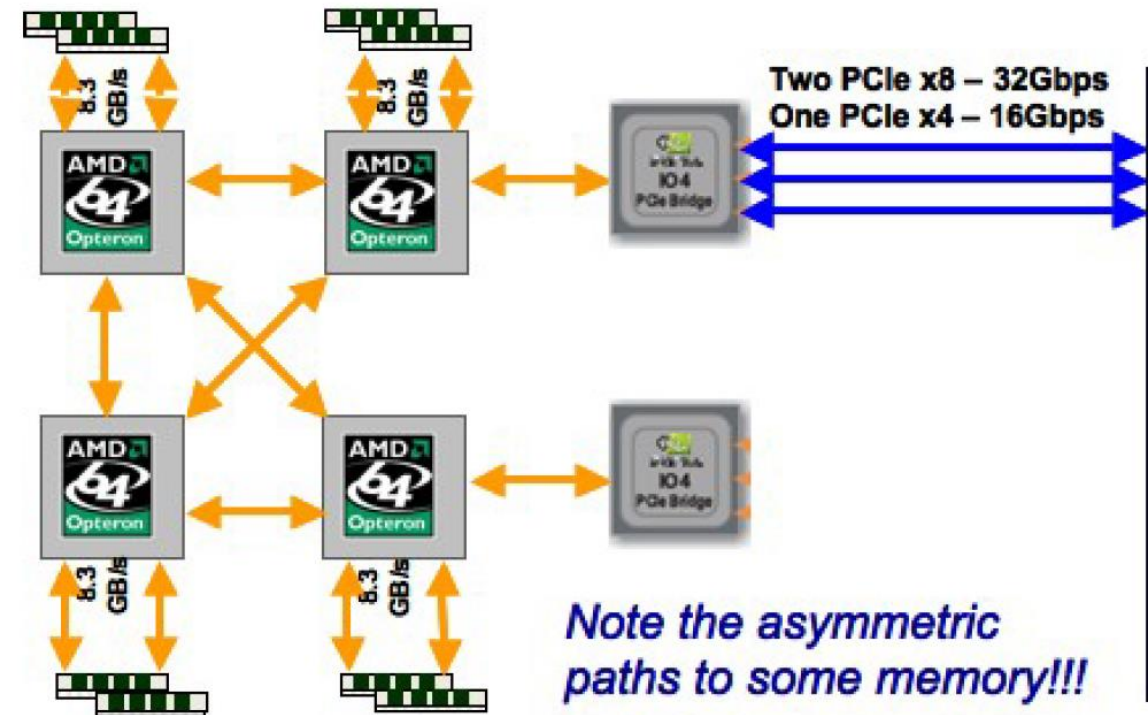
- Multiple processors can operate independently but share the same memory resources.
- Changes in a memory location effected by one processor are visible to all other processors.
- Historically, shared memory machines have been classified as ***Uniform Memory Access (UMA)*** and ***Non-UMA (NUMA)***



Example



- The four-socket motherboard of the Ranger supercomputer.
- Each chip has its own memory (8Gb) but the motherboard acts as if the processors have access to a shared pool of 32Gb.
- Each processor has three connections that could be used to access other memory, but the rightmost two chips use one connection to connect to the network



Numactl --hardware

```
[dadagostino@hpcocapie01 ~]$ numactl --hardware
```

```
available: 1 nodes (0)
```

```
node 0 cpus: 0 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18 19 20 21 22 23 24 25 26 27 28 29 30 31 32
33 34 35 36 37 38 39 40 41 42 43 44 45 46 47 48 49 50 51 52 53 54 55 56 57 58 59 60 61 62 63 64 65 66
67 68 69 70 71 72 73 74 75 76 77 78 79 80 81 82 83 84 85 86 87 88 89 90 91 92 93 94 95 96 97 98 99 100
101 102 103 104 105 106 107 108 109 110 111 112 113 114 115 116 117 118 119 120 121 122 123 124 125 1
26 127 128 129 130 131 132 133 134 135 136 137 138 139 140 141 142 143 144 145 146 147 148 149 150 151
152 153 154 155 156 157 158 159 160 161 162 163 164 165 166 167 168 169 170 171 172 173 174 175 176 1
77 178 179 180 181 182 183 184 185 186 187 188 189 190 191 192 193 194 195 196 197 198 199 200 201 202
203 204 205 206 207 208 209 210 211 212 213 214 215 216 217 218 219 220 221 222 223 224 225 226 227 2
28 229 230 231 232 233 234 235 236 237 238 239 240 241 242 243 244 245 246 247 248 249 250 251 252 253
254 255
```

```
node 0 size: 98180 MB
```

```
node 0 free: 87172 MB
```

```
node distances:
```

```
node 0
```

```
0: 10
```

SMP - UMA

numactl is a utility which can be used to control NUMA policy for processes or shared memory. NUMA (stands for Non-Uniform Memory Access) is a memory architecture in which a given CPU core has variable access speeds to different regions of memory.

Typically, each core will have a region of memory attached to it directly which it can access quickly (local memory), while access to the rest of the memory is slower (non-local memory).

```
[[dago@cluster2 ~]$ numactl --hardware
```

```
available: 2 nodes (0-1)
```

```
node 0 cpus: 0 1 2 3 4 5 12 13 14 15 16 17
```

```
node 0 size: 16374 MB
```

```
node 0 free: 7956 MB
```

```
node 1 cpus: 6 7 8 9 10 11 18 19 20 21 22 23
```

```
node 1 size: 16384 MB
```

```
node 1 free: 3660 MB
```

```
node distances:
```

```
node 0 1
```

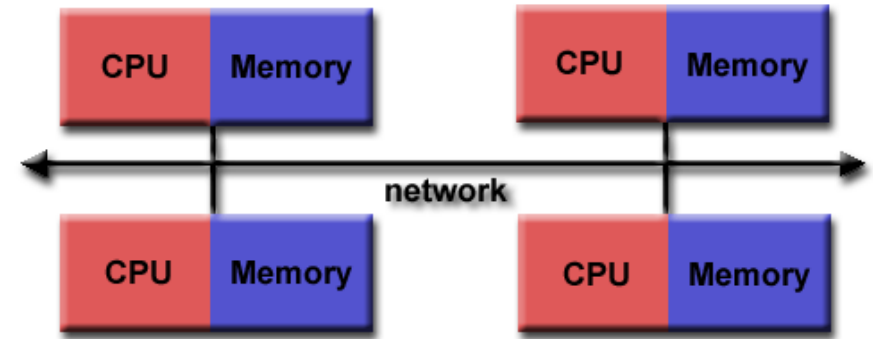
```
0: 10 21
```

```
1: 21 10
```

NUMA

Memory Access – Distributed Memory

- Processors have their own local memory.
- Memory addresses in one processor do not map to another processor, so there is NO concept of global address space across all processors.
- Because each processor has its own local memory, it operates independently. Changes it makes to its local memory have no effect on the memory of other processors. Hence, the concept of cache coherency does not apply.
- When a processor needs access to data in another processor, it is usually the task of the programmer to explicitly define how and when data is communicated.
- Synchronization between tasks is likewise the programmer's responsibility.
- The network "fabric" used for data transfer varies widely, though it can be as simple as Ethernet.



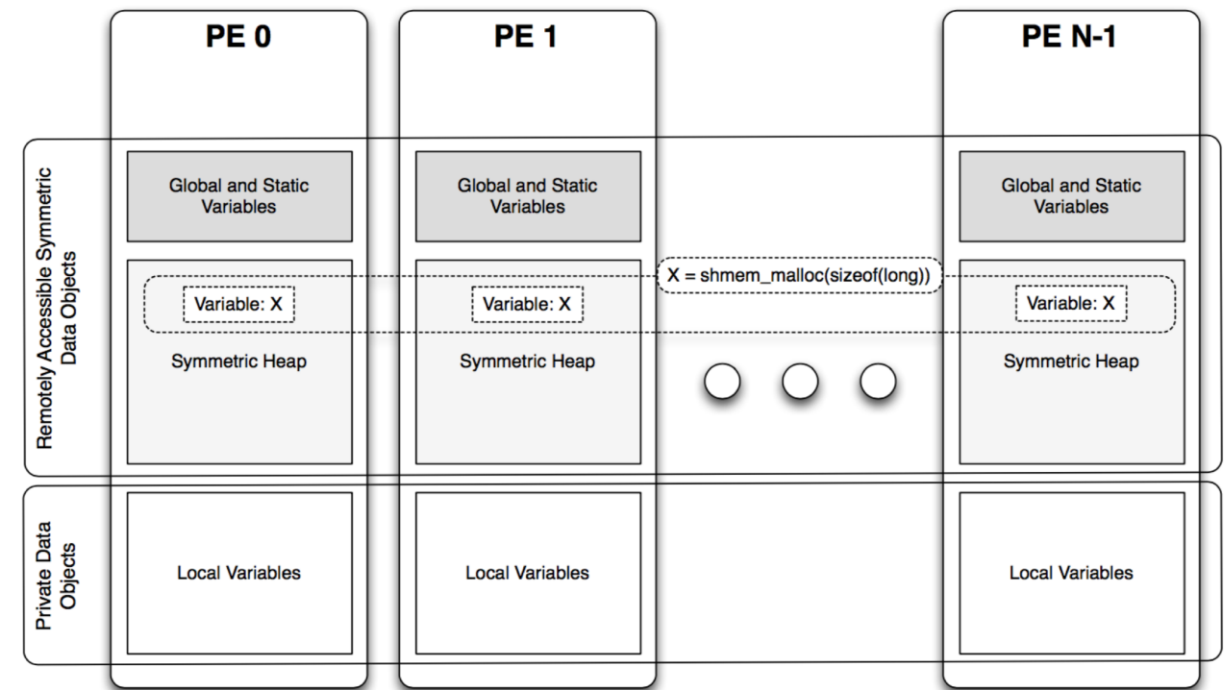
But

Via software is possible to offer a NUMA shared memory.

Partitioned Global Address Space (PGAS) programming models offer HPC programmers an abstracted shared address space, which simplifies programming, while exposing data/thread locality to enhance performance. This can facilitate the development of productive programming languages that can reduce the time to solution, i.e. both development time and execution time.

<http://www.openshmem.org/site/Welcome>

<https://developer.nvidia.com/nvshmem>



Languages

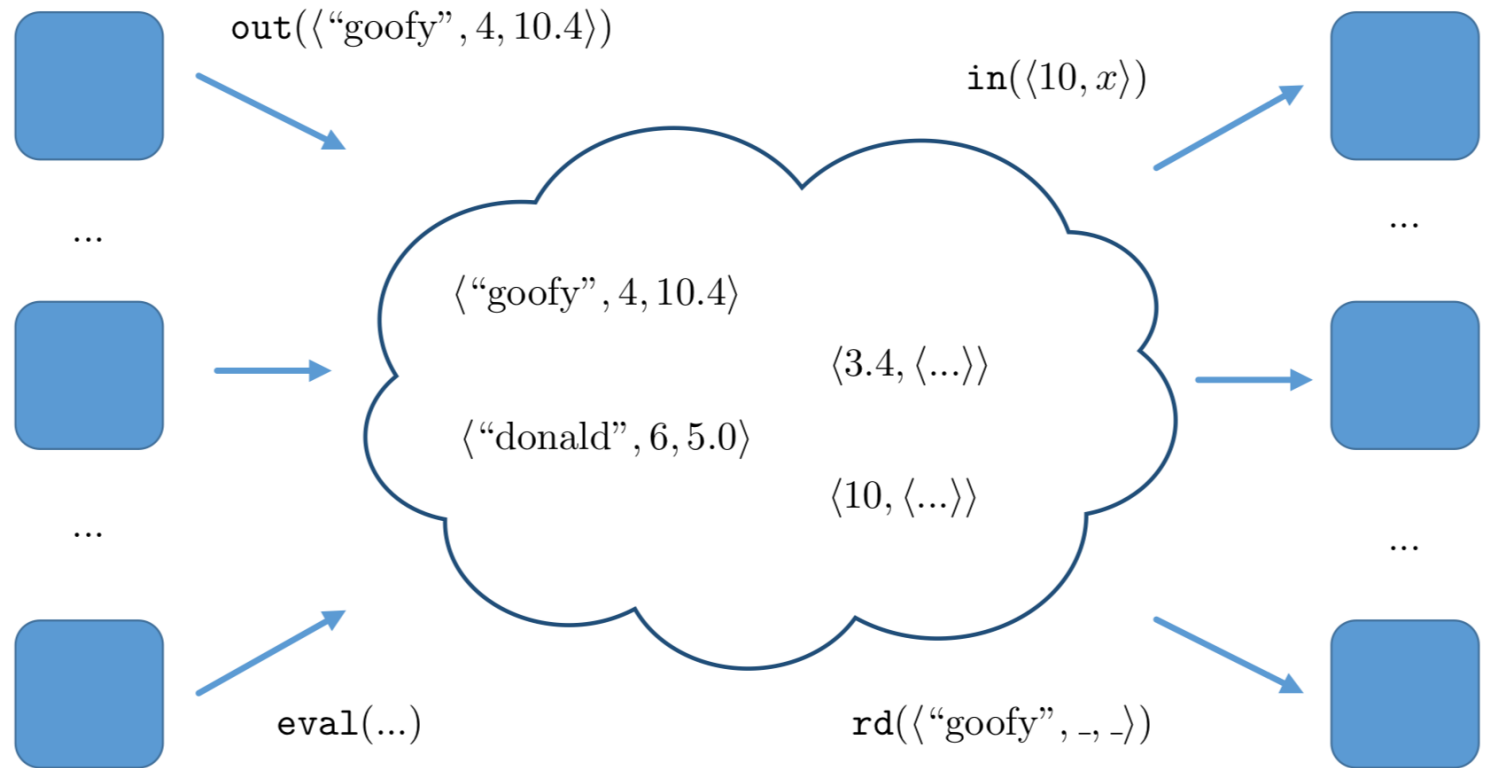
- Unified Parallel C (UPC)
- Co-Array Fortran (CAF)
- X10
- **Chapel**
- STAPL
- Titanium

Libraries

- UPC++
- **OpenSHMEM**
- Global Arrays
- **DASH**
- ...

Or Tuple spaces

- A **tuple space** is a distributed repository, where processes can add, withdraw or read **tuples** by means of atomic operations.
- **Tuples** may contain different values, and processes can inspect their content via pattern matching.
- The lack of a free reference **implementation** for this paradigm has prevented its widespread.

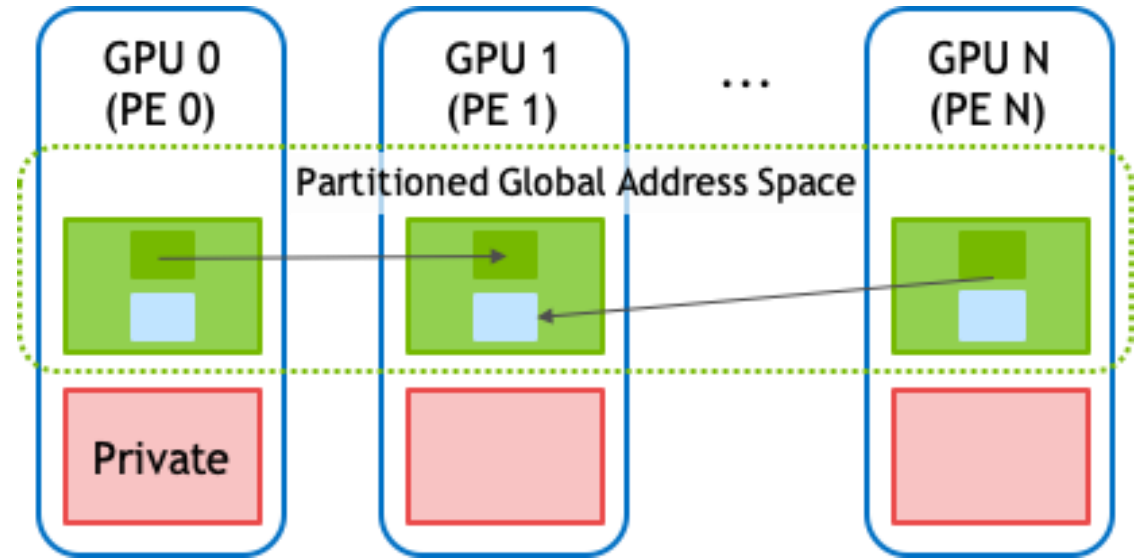
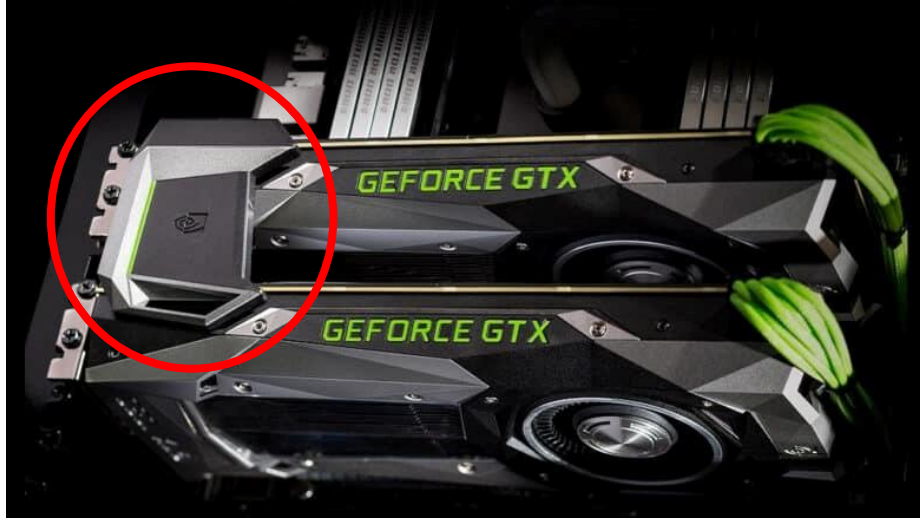


https://en.wikipedia.org/wiki/Tuple_space

<http://wiki.c2.com/?TupleSpace>

<https://arxiv.org/pdf/1612.02979.pdf>

Nvshmem



- NVSHMEM is based on OpenSHMEM, which provides a global address space for data that spans the memory of multiple GPUs.
- An NVSHMEM job consists of several operating system processes, that are referred to as processing elements (PEs).
- An object allocated with ``nvshmem_malloc()`` is called a symmetric data object. Every symmetric data object has a corresponding data object with the same name, type, and size on all PEs.

To summarize

Shared memory

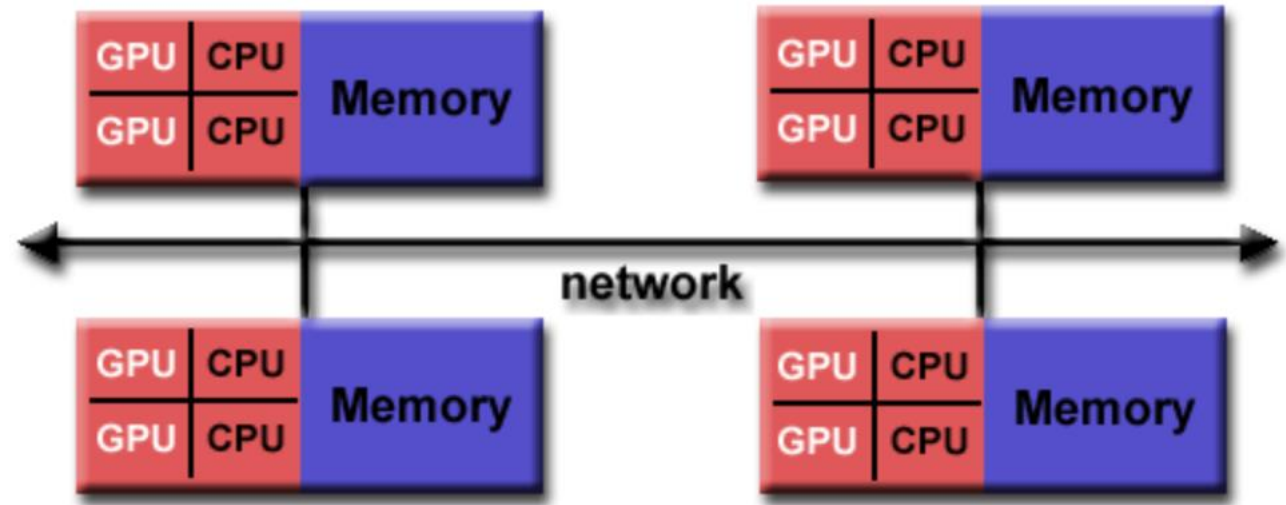
- Advantages:
 - Easier to program
 - Useful for applications with irregular data access patterns (e.g., graph algorithms)
- Disadvantages:
 - The programmer must take care of race conditions
 - Limited memory bandwidth

Distributed memory

- Advantages:
 - Highly scalable, provide very high computational power by adding more nodes
 - Useful for applications with strong locality of reference, with high computation / communication ratio
- Disadvantages:
 - Latency of interconnect network
 - Difficult to program

Actual systems

- The largest and fastest computers in the world today employ both shared and distributed memory architectures.
- The shared memory component can be a shared memory machine and/or graphics processing units (GPU), the node.
- The distributed memory component is the networking of multiple nodes. Therefore, **network communications** are required to move data from one machine to another.
- Current trends seem to indicate that this type of memory architecture will continue to prevail and increase at the high end of computing for the foreseeable future.
- Increased scalability is an important advantage
- Increased programmer complexity is an important disadvantage

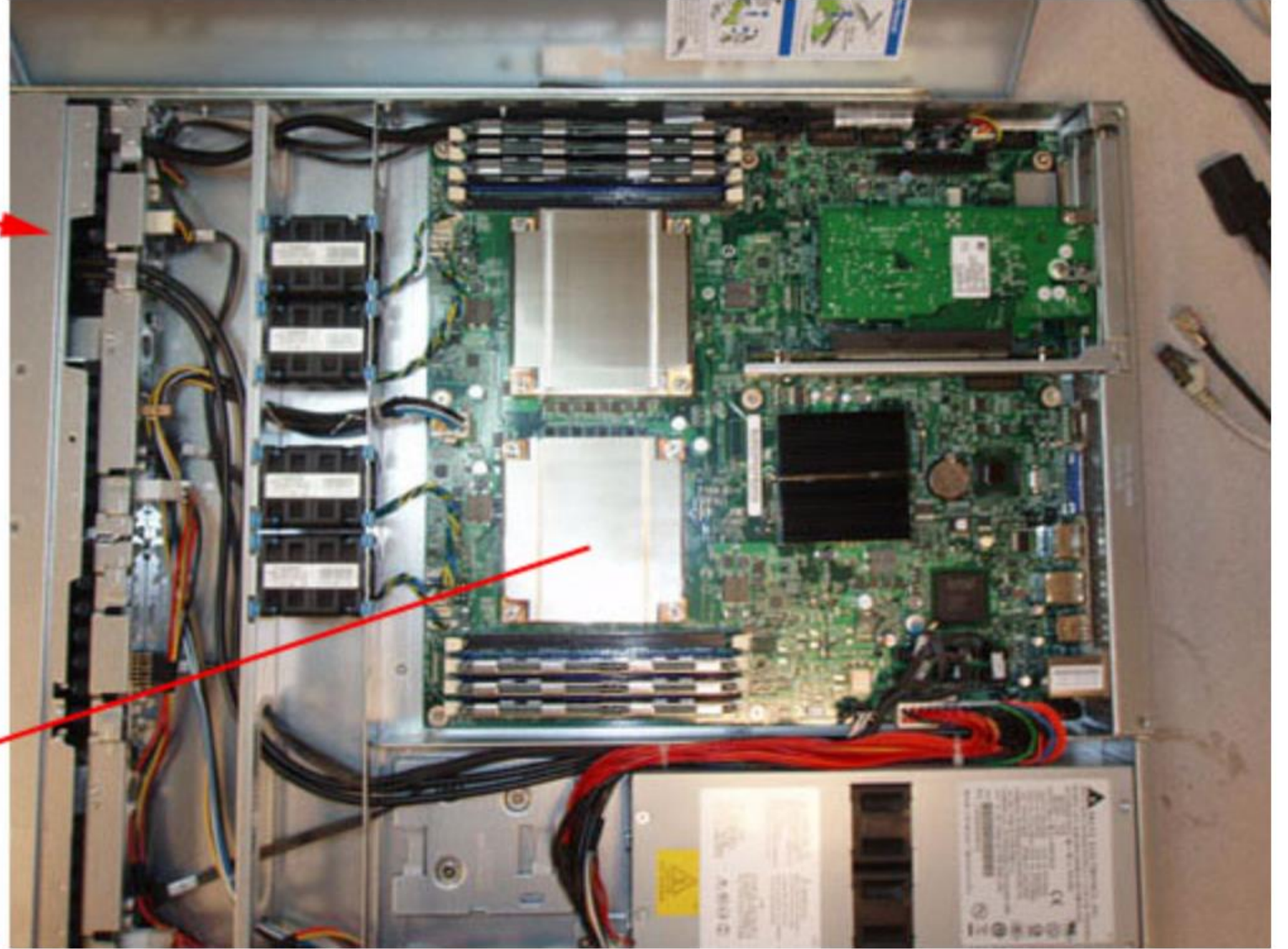
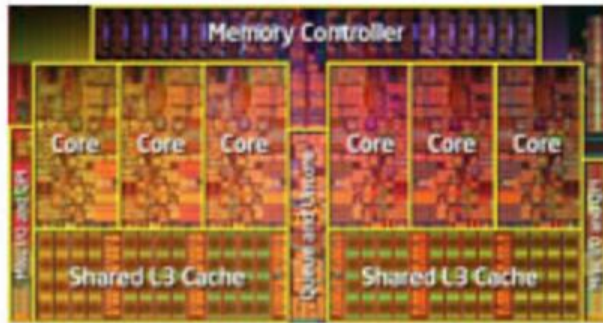




Supercomputer - each blue light is a node

Node - standalone
Von Neumann computer

CPU / Processor / Socket - each
has multiple cores / processors.

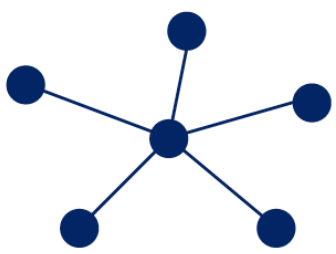


Interconnection

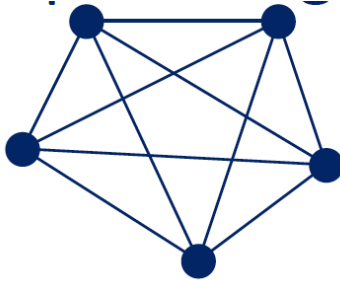
- If a number of processors are working together on a single task, most likely they need to communicate data.
- Here a brief overview on possible interconnection schema
- (Original) Ethernet is a connection scheme where all machines on a network are on a single cable. Simple but limited capacity solution, channel sharing problems...
- In a fully connected configuration, each processor has one wire for the communications with each other processor. This scheme is perfect in the sense that messages can be sent in the minimum amount of time, and two messages will never interfere with each other. Complex to be implemented (and very expensive)..

<http://www.prace-ri.eu/best-practice-guide-modern-interconnects>

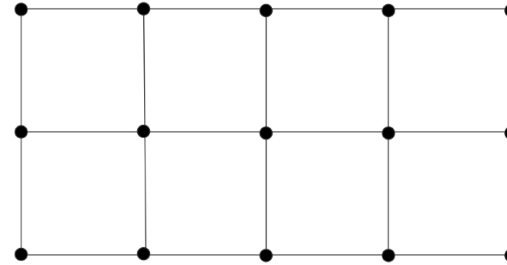
Interconnections



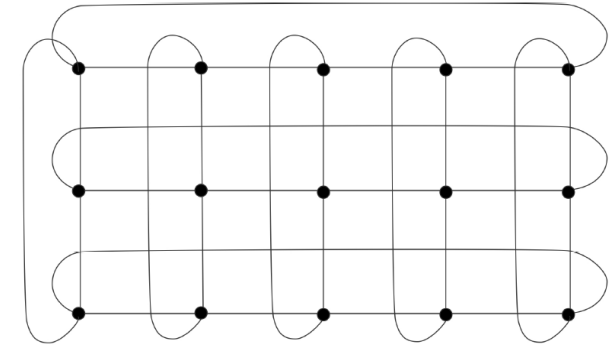
Star



Complete connection

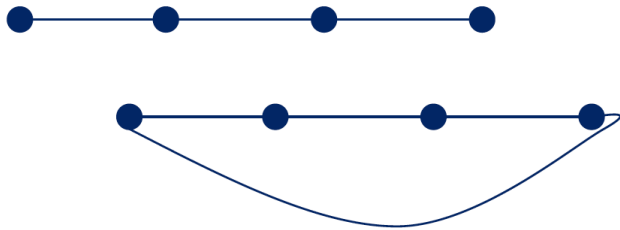


2D Grid

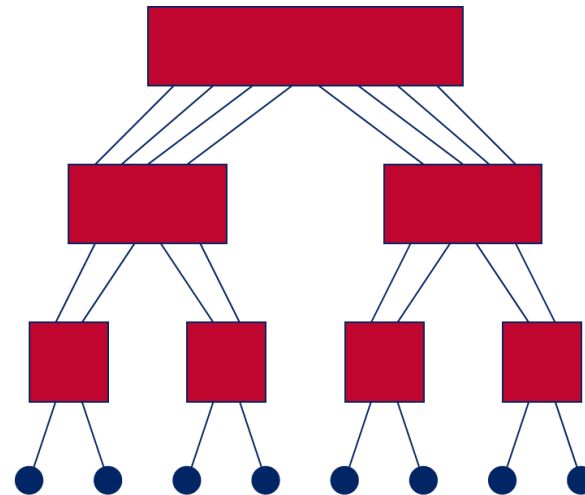


Torus

Linear Array (Ethernet)

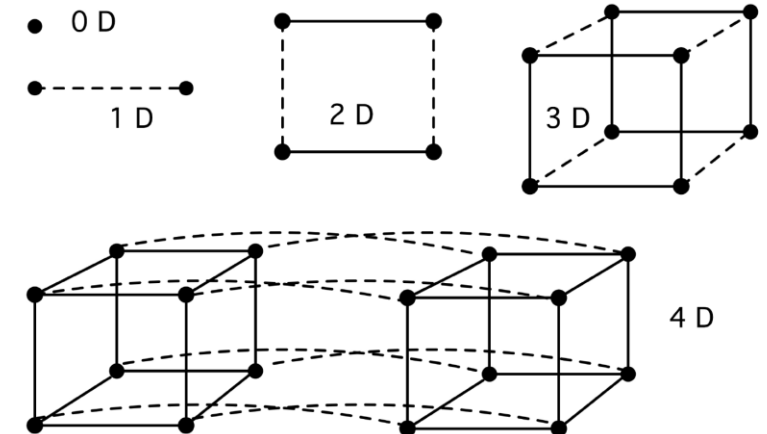


Ring



Fat trees

Hypercubes



Graph theory concepts

Degree

- How many links to other processors does each node have?
- More is better, but also expensive and hard to engineer

Diameter

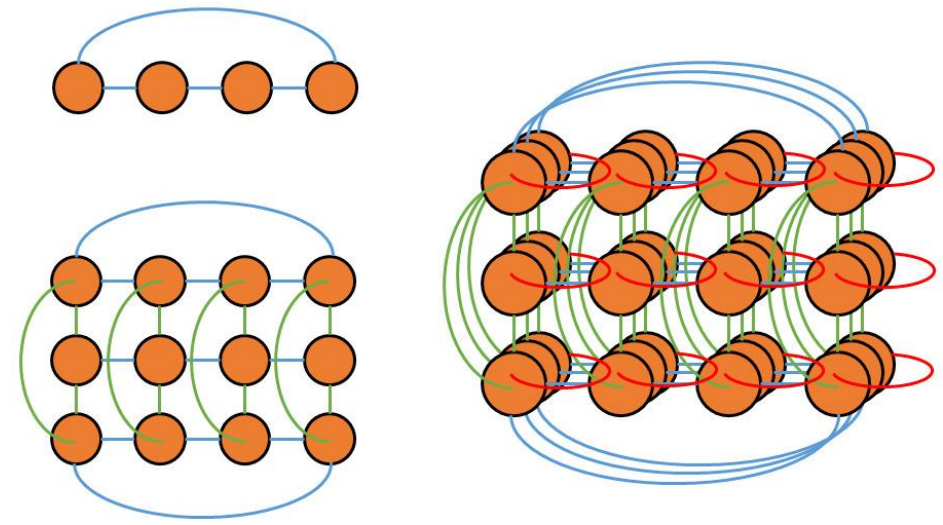
- maximum distance between any two processors in the network:
completely connected network is 1, for star network is 2, for ring is $p/2$ (for p even processors)

Connectivity

- measure # arcs that must be removed to break the connection: 1 for linear array/star/torus (but), 2 for ring/mesh, 4 for torus

Torus

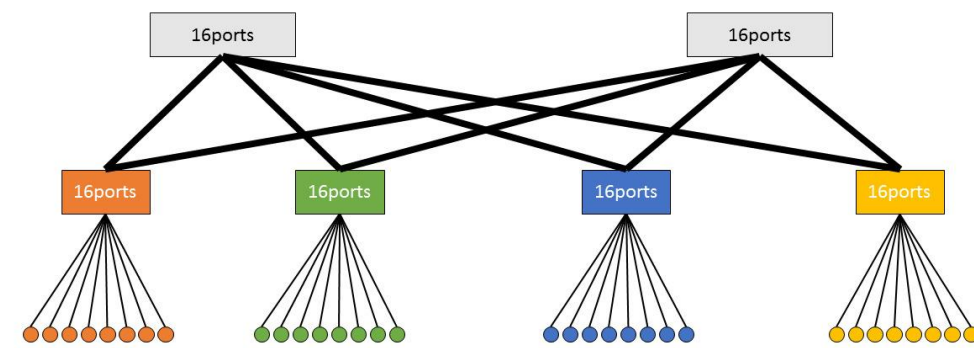
- A torus allows for very fast neighbour-to-neighbour communication. In addition, there can be different paths for moving data between two different nodes.
- The latency is normally quite low and can be distinguished by the maximum number of hops needed.
- As there is no direct central element, the number of individual wires can be very high.
- In an N-dimensional torus, each node has $2N$ individual connections.



Hypercubes and meshes

- It is a fairly natural idea to have 2D or 3D networks, since the world around us is three-dimensional, and computers are often used to model real-life phenomena.
- Hypercubes: smaller diameter mesh, useful for broadcasts
- The main difference between a pure hypercube and a torus are the number of connections of the border nodes. Within the torus setup each network node has the same number of neighbour nodes, while in the hypercube only the inner nodes have the same number of neighbours and the border nodes have fewer neighbours. This allows for fewer required network hops in a torus network if border nodes need to communicate.

Fat Trees



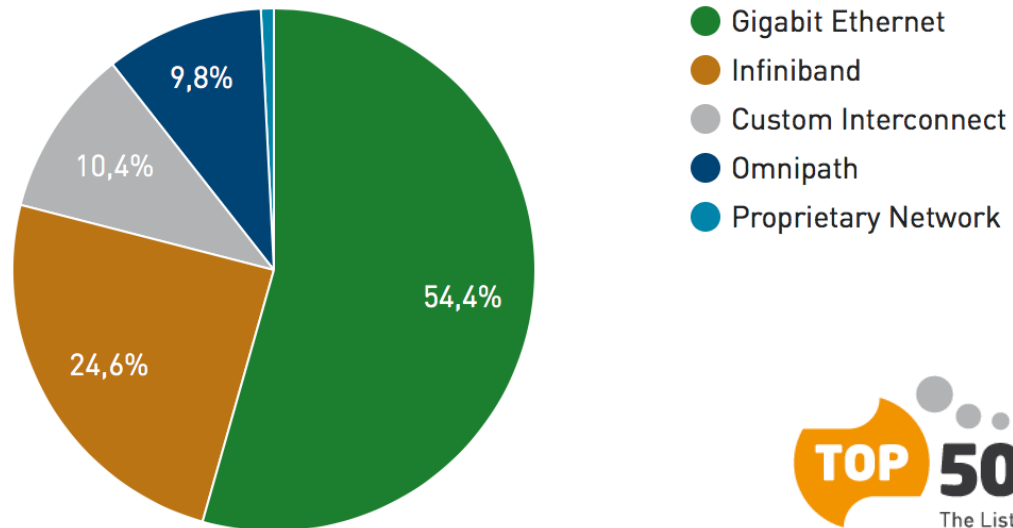
- Are used to provide a low latency network between all involved nodes.
- All computing nodes are located on the leaves of the tree structure and switches are located on the inner nodes.
- The idea of the fat tree is, instead of using the same wire “thickness” for all connections, to have “thicker” wires closer to the top of the tree and provide a higher bandwidth over the corresponding switches.
- In an ideal case a fat-tree is a tree network where each level has the same total bandwidth, so that congestion problem does not occur.

Infiniband and High Performance Ethernet

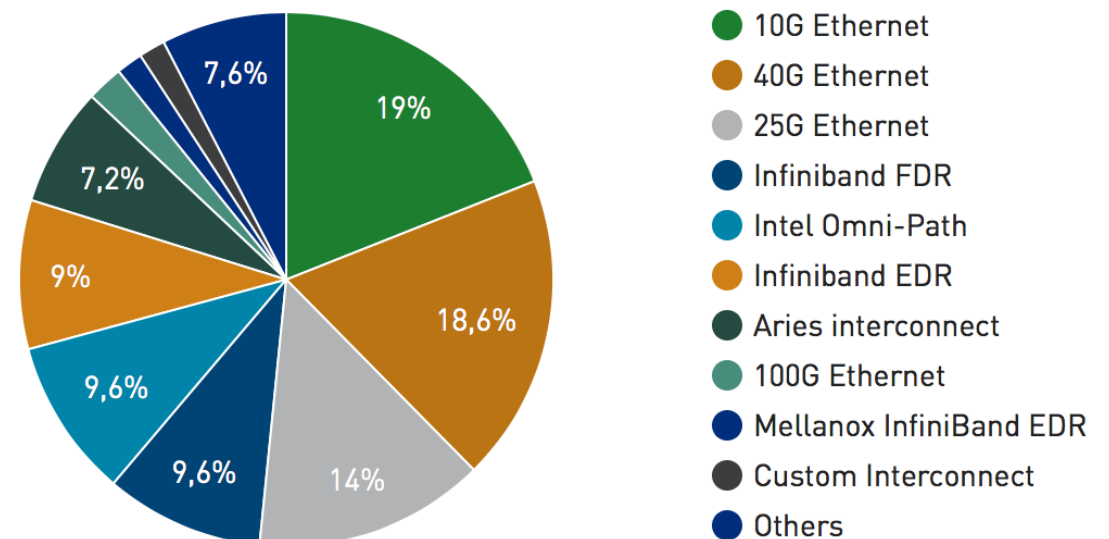
<http://prace.it4i.cz/sites/prace.it4i.cz/files/files/iohed-01-2018-slides.pdf>

<https://en.wikipedia.org/wiki/InfiniBand>

Interconnect Family System Share



Interconnect System Share



Network Speed Acceleration with IB and HSE

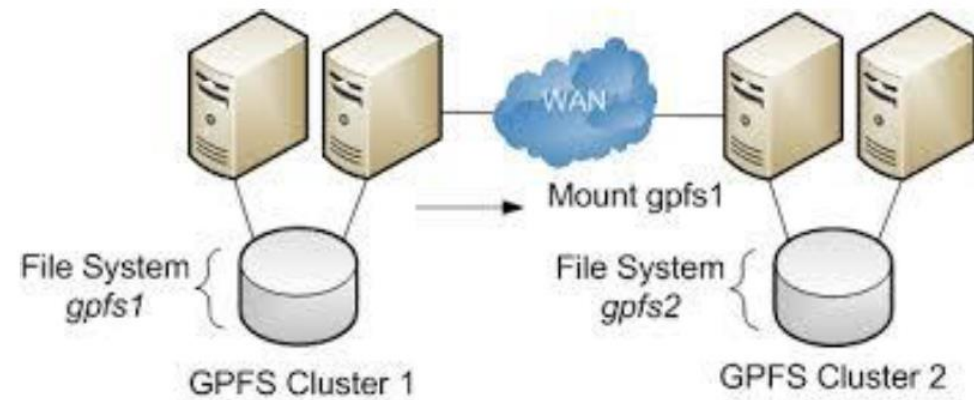
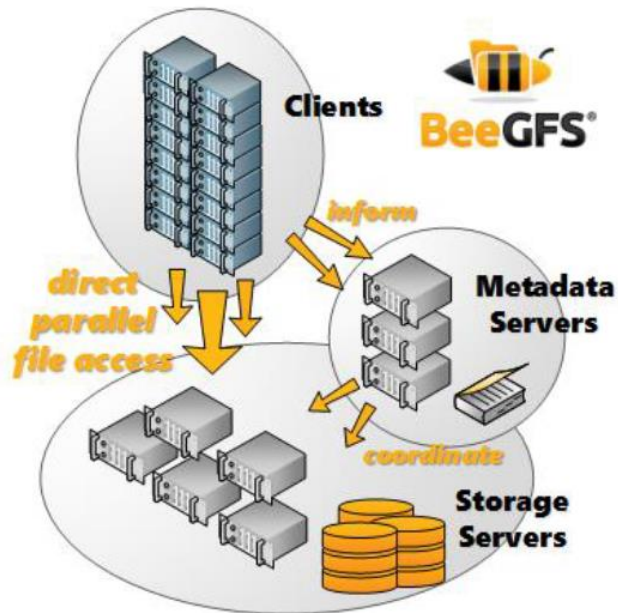
Ethernet (1979 -)	10 Mbit/sec
Fast Ethernet (1993 -)	100 Mbit/sec
Gigabit Ethernet (1995 -)	1000 Mbit /sec
ATM (1995 -)	155/622/1024 Mbit/sec
Myrinet (1993 -)	1 Gbit/sec
Fibre Channel (1994 -)	1 Gbit/sec
InfiniBand (2001 -)	2 Gbit/sec (1X SDR)
10-Gigabit Ethernet (2001 -)	10 Gbit/sec
InfiniBand (2003 -)	8 Gbit/sec (4X SDR)
InfiniBand (2005 -)	16 Gbit/sec (4X DDR)
	24 Gbit/sec (12X SDR)
InfiniBand (2007 -)	32 Gbit/sec (4X QDR)
40-Gigabit Ethernet (2010 -)	40 Gbit/sec
InfiniBand (2011 -)	54.6 Gbit/sec (4X FDR)
InfiniBand (2012 -)	2 x 54.6 Gbit/sec (4X Dual-FDR)
25-/50-Gigabit Ethernet (2014 -)	25/50 Gbit/sec
100-Gigabit Ethernet (2015 -)	100 Gbit/sec
Omni-Path (2015 -)	100 Gbit/sec
InfiniBand (2015 -)	100 Gbit/sec (4X EDR)
InfiniBand (2016 -)	200 Gbit/sec (4X HDR)

100 times in the last 15 years

Parallel filesystems

The filesystem manages how files are stored on disks and how they can be retrieved or written.

In a parallel architecture, with many simultaneous accesses to the disks, important to use a *parallel filesystem* technology such as GPFS, LUSTRE, BeeGFS etc.



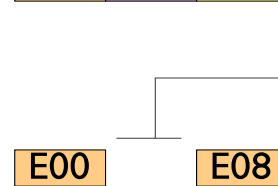
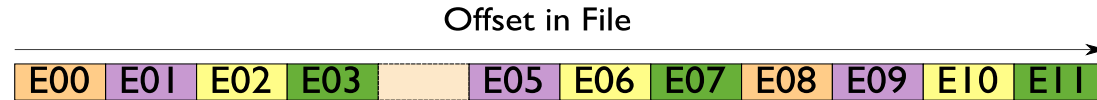
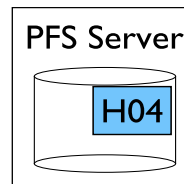
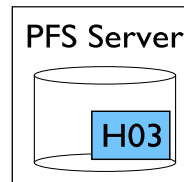
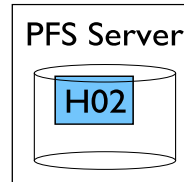
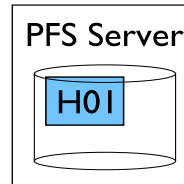
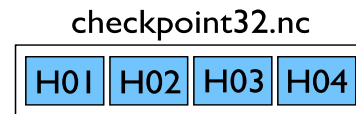
Data Distribution in Parallel File Systems

Distribution across multiple servers allows concurrent access.

Logically a file is an extendable sequence of bytes that can be referenced by offset into the sequence.

Metadata associated with the file specifies a mapping of this sequence of bytes into a set of objects on PFS servers.

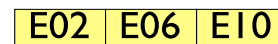
Extents in the byte sequence are mapped into objects on PFS servers. This mapping is usually determined at file creation time and is often a round-robin distribution of a fixed extent size over the allocated objects.



Space is allocated on demand, so unwritten "holes" in the logical file do not consume disk space.



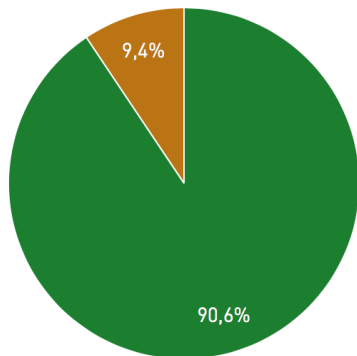
A static mapping from logical file to objects allows clients to easily calculate server(s) to contact for specific regions, eliminating need to interact with a metadata server on each I/O operation.



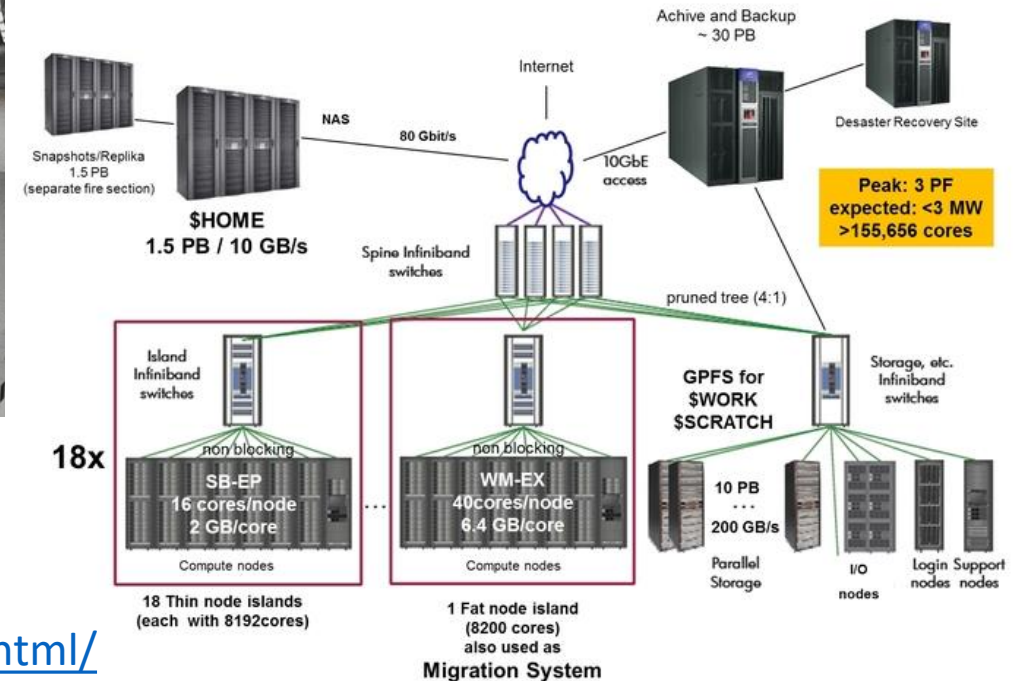
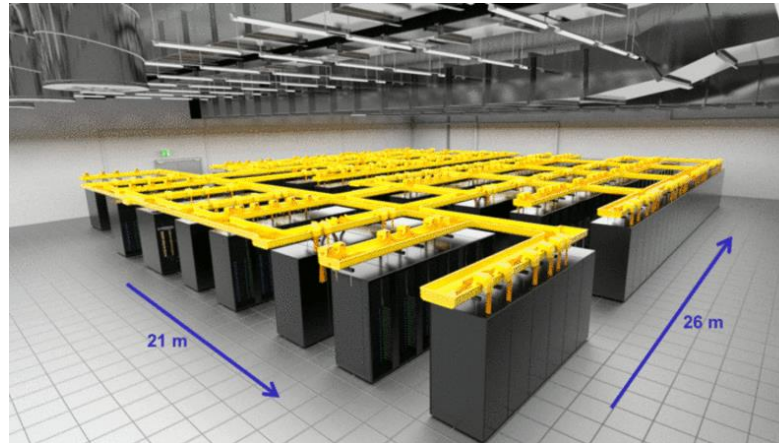
Clusters

- Parallel computer systems comprising an integrated collection of independent nodes, each of which is a system in its own right, capable of independent operation and derived from products developed and marketed for other stand-alone purposes
- E.g. SuperMUC

Architecture System Share



● Cluster
● MPP

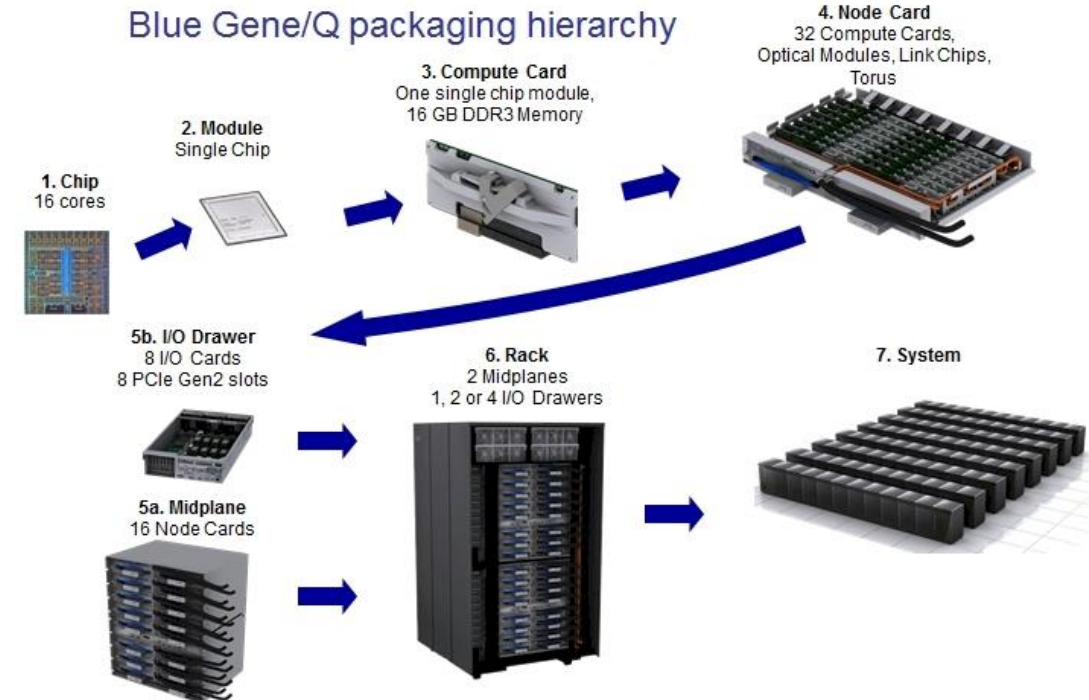


<https://doi.org/10.1109/MCSE.2005.34>

<http://www.prace-ri.eu/best-practice-guide-supermuc-html/>

Massively Parallel Processors

- These are more tightly-integrated systems: individual nodes cannot run on their own and they are connected by a custom network (like a multidimensional torus).
- But, similarly to a cluster, there is no single, shared memory spanning all the nodes
- The five-dimensional torus (10 links) is the Blue Gene/Q internal network used for all MPI communications and also for all the I/O between the compute nodes and the I/O nodes.



Linux Cluster

- An LC production cluster has four types of nodes, based upon function, which can differ in configuration details:
 - Login, Interactive/debug (with queues) , Batch, I/O and service nodes (not for users)

<https://wiki.u-gov.it/confluence/display/SCAIUS/UG3.1%3A+MARCONI+UserGuide#UG3.1:MARCONIUserGuide-Productionenvironment>

- **Login nodes:**

- Every system has a designated number of login nodes - depends upon the size of the system. Some examples:
 - agate = 2
 - sierra = 5
 - quartz = 14
 - zin = 20
- Login nodes are shared by multiple users
- Primarily used for interactive work such as editing files, submitting batch jobs, compiling, running GUIs, etc.
- Interactive use exclusively - login only nodes do not permit any batch jobs.
- DO NOT run production jobs on login nodes! Remember, you are sharing login nodes with other users.

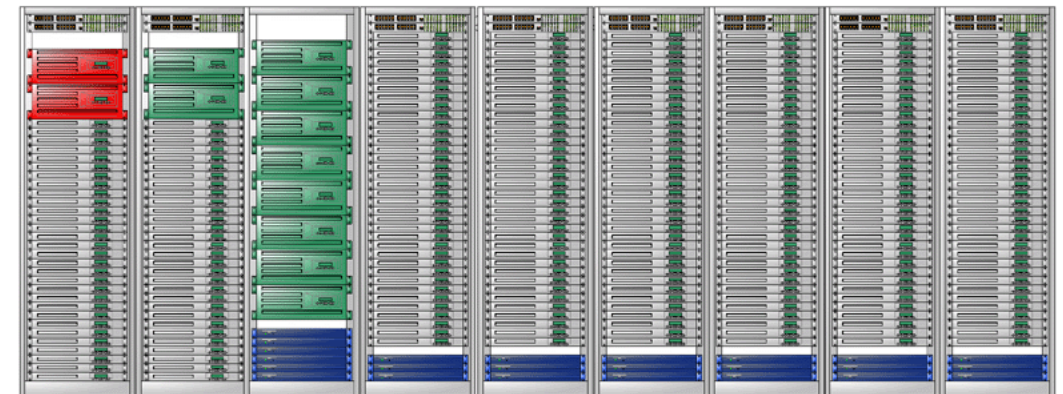
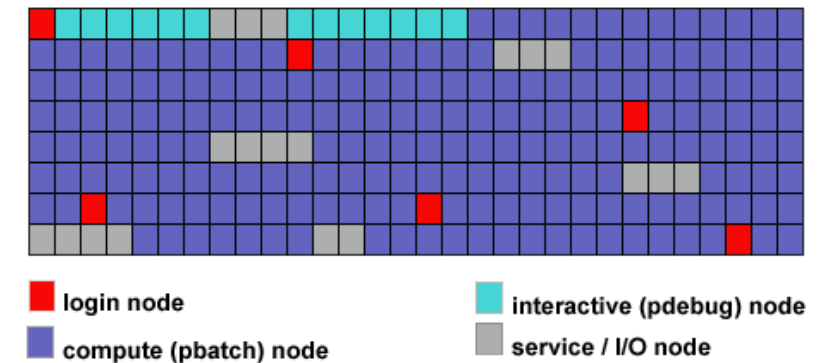
- **Interactive/debug (pdebug) nodes:**

- Most LC systems have nodes that are designated for interactive work.
- Meant for testing, prototyping, debugging, and small, short jobs
- Cannot be logged into unless you already have a job running on them
- Nodes run one job at a time - not shared like login nodes
- Can also be used through the batch system

- **Batch (pbatch) nodes:**

- Comprise the majority of nodes on each system
- Meant for production work
- Work is submitted via a batch scheduler (Slurm, Moab)
- Cannot be logged into unless you already have a job running on them
- Nodes run one job at a time - not shared like login nodes

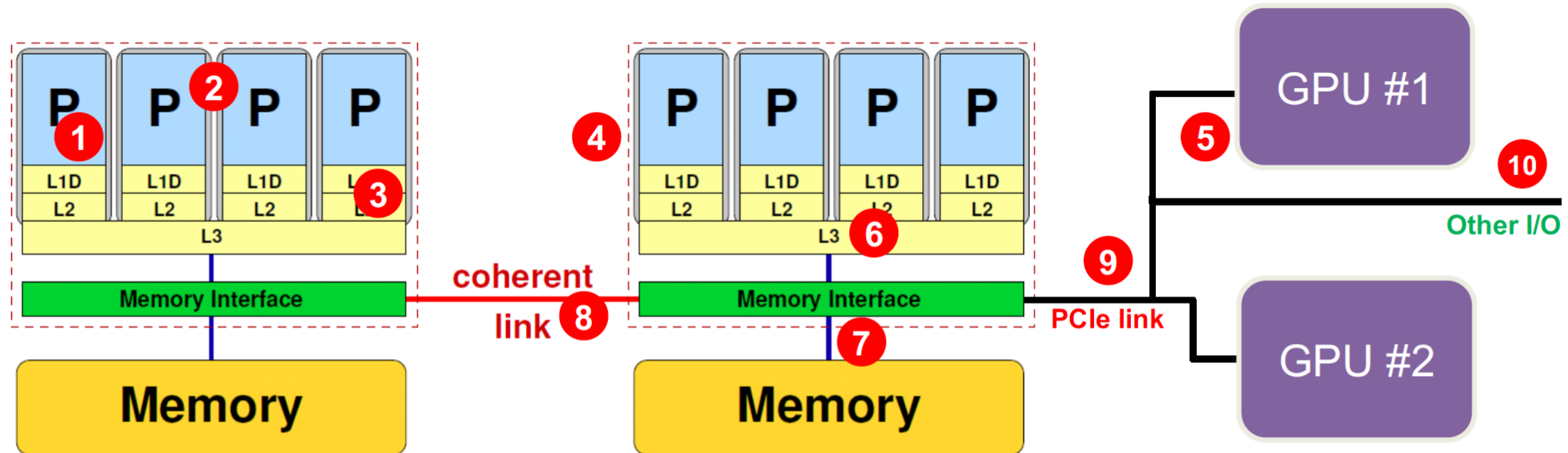
Typical LC System



https://computing.llnl.gov/tutorials/linux_clusters

Parallelism in a modern computer node

Parallel and shared resources within a shared-memory node



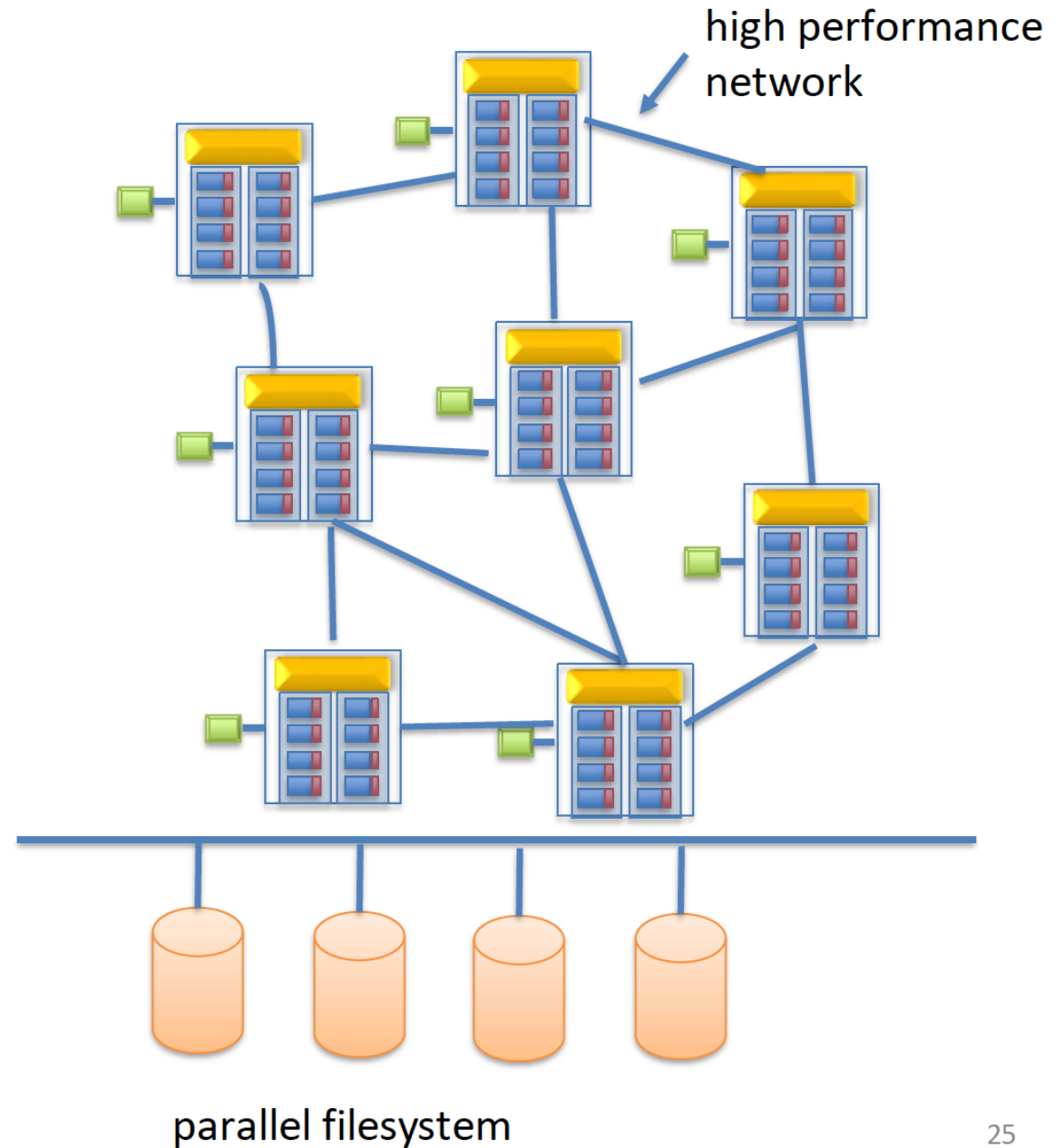
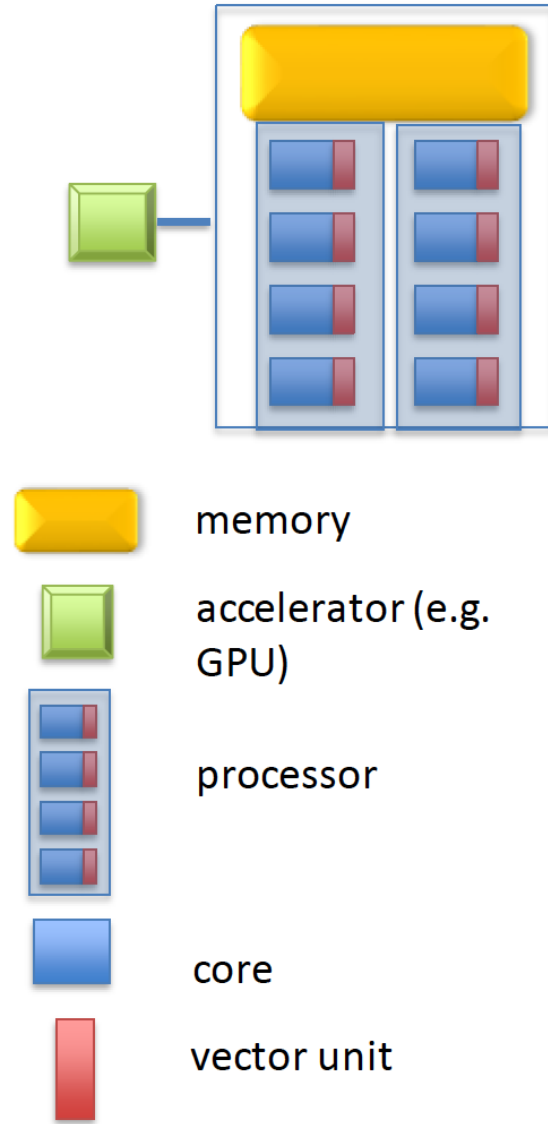
Parallel resources:

- Core 1
- CPU 2
- Inner cache levels 3
- Sockets / ccNUMA domains 4
- Multiple accelerators 5

Shared resources:

- Outer cache level per socket 6
- Memory bus per socket 7
- Intersocket link 8
- PCIe bus(es) 9
- Other I/O resources 10

Putting it all together



Which factors drive the evolution in HPC architecture?

- ❑ The first (super) computers were mainly used by defence organisations and the US Govt (esp. Department of Energy) still makes significant investments. Later they were used for scientific research in a small number of centres.
- ❑ But the high cost of the dedicated components, and the fact that **HPC is not a strong market**, has caused a shift into using commodity or off-the-shelf devices such as processors, disks, memories, networks, etc.
- ❑ This shift has had a number of consequences:
 - ❑ Some manufactures have changed business or no longer make supercomputers (e.g. SUN microsystems).
 - ❑ Other supercomputer vendors (e.g. CRAY and SGI) no longer make microprocessors so the market is dominated by one or two brands, i.e. Intel and, to a lesser extent, IBM PowerPC.
 - ❑ Since microprocessors were not designed for HPC, the programmer must work harder to get maximum performance.
- ❑ But porting has become easier as only a few processor types are available and Linux has replaced all the other operating systems. It is now also possible for smaller organisations such as university departments to run small clusters.